Secure Microfinance and Welfare Management Platform Using Blockchain and AI

INDIVIDUAL THESIS

A.I. Jayasinghe

IT22552310

A Multilingual RAG Based Conversational Assistant for Improving Access to Welfare and Micro Loan Information

Group ID: R26-IT-017

Report submitted in partial fulfilment of the requirements for the

Bachelor of Science (Honors) in Information Technology

Specialization in Information Technology Department of

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: A.I.Jayasinghe

Student ID: IT22552310

Signature: _____ Date: _____

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor:		
Co-supervisor:		

ABSTRACT

The SmartGrama AI Assistant tries to make welfare programme and micro-loan information easier to reach by giving people a chat interface that works in English and Sinhala. Old ways of getting this information can be hard because people must read papers learn fancy words or look in many places. These problems show that we need a tool that can fetch the right, details and show them in a simple friendly way.The goal of this research part was to create, build and test a Natural Language Processing (NLP) system that lets people ask questions in the language they like and get answers that fit the situation from a knowledge base.The part was made with a React end and a Flask/Python back end. Sentence Transformers made embeddings. Facebook AI Similarity Search (FAISS) helped pull the similar knowledge parts fast. Those parts were put into a Retrieval-Augmented Generation (RAG) process, where an Ollama-hosted large language model made replies using the given domain context. A deep translator tool handled conversion between English and Sinhala when needed.The test set had 217 questions and kept track of the generated replies the correct answers, the pulled context, language, category and how long the reply took. This test let us look at how the system fetches and answers questions especially when there is not context wrong words or missing procedure details.The results show how crucial good quality knowledge bases, good passage picking and answers that are backed by evidence are for building a trustworthy domain-specific chat assistant.My own work was on designing, building, linking and testing the NLP and RAG pipeline inside the SmartGrama prototype.The final solution shows a way to give people in many languages access, to welfare and micro-loan info. Still we need retrieval accuracy, better translation and more reliable answers before it can be used in real public services.

Keywords: Natural Language Processing, Retrieval Augmented Generation, Semantic Search, Multilingual Conversational AI, FAISS

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my project supervisor Dr Dharshana Kasthurirathne and Co-Supervisor Ms Chathurya Kumarapperuma for their continuous guidance, valuable suggestions, and support throughout the development of this research project.

I would also like to extend my appreciation to director Mr Rangith Disanayaka of Co operative Society Battaramulla, Manager of Samupakara Gramiya Bank Battaramulla and Samurdhi Bank for providing access to loan-related data, Samurdhi Welfare related data and requirement specifications that enabled the practical development and evaluation of the Microfinance and welfare management platform.

My sincere thanks also go to the faculty and staff members of the Sri Lanka Institute of Information Technology for providing the necessary academic support and resources during the research. Finally, I would like to thank our friends and families for their encouragement and support during the completion of this project.

CONTENTS

ABSTRACT 3

ACKNOWLEDGEMENTS 4

LIST OF FIGURES 8

LIST OF TABLES 8

LIST OF ABBREVIATIONS 10

1 INTRODUCTION 11

1.1 Overall Project Context 11

1.2 Individual Component Context 12

1.3 Problem Statement 13

1.4 Research Gap 14

1.5 Aim and Objectives 15

1.6 Research Questions 16

1.7 Scope, Limitations, and Assumptions 16

1.8 Individual Contribution 17

1.9 Thesis Structure 18

2.LITERATURE REVIEW AND RELATED WORK 19

2.1 Review Method and Scope 19

2.2 Domain and Theoretical Foundations 20

2.3 Related Research 26

2.4 Existing Systems, Products, and Techniques 30

2.5 Gap Synthesis and Conceptual Framework 34

3: RESEARCH METHODOLOGY 37

3.1 Research Design 37

3.2 Data, Participants or Study Materials 38

3.3 Experimental or Study Procedure 40

3.4 Measures and Evaluation Metrics 44

3.5 Data Analysis 47

3.6 Ethics, Privacy, Security and Responsible AI 49
3.7 Reproducibility and Research Artefacts 51
4 PRODUCT AND SOLUTION DESIGN 52
4.1 Requirements 52
4.2 Component Architecture Within the Complete Solution 54
4.3 Design Alternatives and Decisions 55
4.4 Detailed Design 57
4.5 Standards and Professional Practice 60
5.IMPLENTATION INTEGRATION AND TESTING 62
5.1 Development environment and technology stack 62
5.2 Component Implementation 64
5.3 Integration with the Other Components 67
5.4 Verification and Software Testing 69
5.5 Deployment and Operation 72
6: RESULTS AND VALIDATION 74
6.1 Research Results 74
6.2 Product and Technical Results 78
6.3 User or Stakeholder Validation 79
6.4 Integration Results 80
6.5 Summary of Findings 81
7: DISCUSSION 82
7.1 Interpretation of Findings 82
7.2 Comparison with Literature, Baselines, and Existing Solutions 85
7.3 Research Contribution 87
7.4 Product Contribution 88
7.5 Limitations and Threats to Validity 88
7.6 Implications 92

8: COMMERCIALISATION AND SUSTAINABILITY 93
8.1 Value Proposition and Target Users 93
8.2 Market and Competitive Position 95
8.3 Business or Sustainability Model 97
8.4 Scalability, Intellectual Property and Compliance 100
8.5 Contribution to the Sustainable Development Goals 102
9: CONCLUSION AND FUTURE DIRECTIONS 105
9.1 Conclusion 105
9.2 Achievement of Research Objectives 106
9.3 Individual Research and Product Contributions 108
9.4 Future Directions 109
9.5 Final Conclusion 113
REFERENCES 115

LIST OF FIGURES

Figure 1 Relationship Between the Research Gap and Evaluation 36
Figure 2 SmartGrama NLP Frontend 65
Figure 3 SmartGrama NLP Local Deployment Arrangement 73

LIST OF TABLES

Table 1Summary of Research Problems and Proposed Solutions 12
Table 2 Summary of Related Research on Language Models, Retrieval-Augmented Generation, and Semantic Search for SmartGrama 29
Table 3 Comparison of Conversational and Information Retrieval Approaches 32
Table 4 Technologies Used in the Proposed NLP Architecture 33
Table 5 Identified Research Gaps and the Proposed Response 35
Table 6 Categories of Information Supported by the SmartGrama NLP Assistant 39
Table 7 Software Technologies and Their Roles in the SmartGrama NLP Component 40
Table 8 Software Technologies and Their Roles in the SmartGrama NLP Component 46
Table 9 Research Artefacts and Their Purposes in the SmartGrama NLP Component 51
Table 10 Functional Requirements of the SmartGrama NLP Component 52
Table 11 Non-Functional Requirements and Design Responses 53
Table 12 Architecture Layers and Their Responsibilities 55
Table 13 Design Alternatives and Selected Decisions 57
Table 14 Technology Stack Used in the SmartGrama NLP Component 62
Table 15 Development Environment Requirements 63
Table 16 Integration Points and Responsibilities 69
Table 17 Functional Test Scenarios 70
Table 18 Evaluation Dimensions for the 217 Question Dataset 72
Table 19 Characteristics of the SmartGrama NLP Evaluation Dataset 75
Table 20 Evaluation Dimensions and Evidence 76
Table 21 Product and Technical Results 78

Table 22 Validation Evidence and Its Scope 79
Table 23 Integration Assessment of the NLP Component 80
Table 24 Qualitative Comparison of Conversational and Information Retrieval Approaches 87
Table 25 Summary of Limitations and Recommended Responses 91
Table 26 Potential Target Users and Their Expected Benefits 94
Table 27 Qualitative Competitive Positioning 96
Table 28 Potential Business and Sustainability Models 97
Table 29 Potential Operational Costs 99
Table 30 Intellectual Property and Licensing Considerations 101
Table 31 Potential Alignment of the SmartGrama NLP Component with the SDGs 104
Table 32 Summary of Research Objectives and Evidence of Achievement 108
Table 33 Prioritised Future Development Directions 112

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CPU	Central Processing Unit
FAISS	Facebook AI Similarity Search
GPU	Graphics Processing Unit
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
LLM	Large Language Model
NLP	Natural Language Processing
RAG	Retrieval Augmented Generation

1 INTRODUCTION

1.1 Overall Project Context

The rapid development of Artificial Intelligence (AI), Natural Language Processing (NLP), and digital information systems has created new opportunities to improve the delivery of public and financial services. Modern digital platforms enable organizations to manage information, communicate with users, and provide services more efficiently. However, the accessibility of these platforms remains a challenge for people who experience language barriers, limited digital literacy, and difficulties understanding formal administrative procedures.

In Sri Lanka, welfare assistance and microfinance services play an important role in supporting individuals, families, and small business owners. Citizens may need information regarding welfare programmes, financial assistance, micro-loan eligibility, application procedures, repayment requirements, and supporting documentation. Obtaining this information may require visiting government offices, contacting financial institutions, or searching through different documents and online resources.

These processes can create difficulties for citizens who are unfamiliar with formal administrative terminology or who prefer to communicate in Sinhala rather than English. Furthermore, information distributed across different sources can make it difficult for users to identify the specific information relevant to their circumstances.

The SmartGrama project, titled A Blockchain Integrated Microfinance and Welfare Management Platform, aims to improve the management and accessibility of microfinance and welfare-related services through an integrated digital solution. The overall platform incorporates different technological components to support service delivery and improve the interaction between citizens and relevant institutions.

Within this platform, the Natural Language Processing (NLP) component provides a conversational interface through which users can request information using natural language. Instead of navigating multiple pages or manually searching through documents, users can ask questions and receive understandable responses related to welfare and micro-loan services.

The proposed NLP component combines multilingual language processing with Retrieval-Augmented Generation (RAG). The RAG approach retrieves relevant information from a domain-specific knowledge base before supplying it to a Large Language Model (LLM) for response generation. This architecture is intended to improve the relevance and factual grounding of responses compared with approaches that depend entirely on the language model's pretrained knowledge.

The component is designed to support English and Sinhala interaction. Its wider objective is to make information about welfare and microfinance services more accessible to citizens while supporting the digital transformation goals of the SmartGrama platform.

Table 1 Summary of Research Problems and Proposed Solutions

Identified problem	Proposed solution	Expected benefit
Difficulty locating welfare and micro loan information	Domain-specific knowledge base	More direct access to relevant information
Different wording for similar questions	Semantic embedding and retrieval	Improved retrieval across paraphrased questions
Unsupported language-model responses	RAG-based contextual grounding	Responses supported by retrieved information
English and Sinhala language barriers	Translation-based multilingual processing	More accessible conversational interaction
Difficulty evaluating answer reliability	Structured response evaluation	Evidence-based assessment of component performance

1.2 Individual Component Context

The individual component addressed in this thesis is the Natural Language Processing component of the SmartGrama platform. Its primary responsibility is to enable users to communicate with the system through natural language questions and obtain information relevant to welfare and micro loan services.

Traditional information systems generally require users to navigate structured menus, complete forms, or search for information using specific keywords. Although these methods can be effective for experienced users, they may create additional difficulties for individuals who are unfamiliar with the system or the terminology used in financial and administrative services.

A conversational NLP component provides an alternative method of interaction. Users can express their information needs in their own words, while the system processes the submitted query, retrieves relevant information, and generates a response.

The research paper associated with this component describes an architecture incorporating a React-based frontend, a Flask backend, Sentence Transformers for semantic embeddings, FAISS for vector similarity search, translation functionality through the deep translator library, and a locally deployed language model accessed through Ollama.

The component can be divided into several functional stages.

First, the user submits a question through the conversational interface. The system then processes the query and determines the appropriate language processing pathway. Where necessary, translation is applied to support retrieval and response generation.

Second, the processed query is converted into a semantic vector representation using a sentence-embedding model. This representation enables the retrieval subsystem to identify knowledge base segments that are semantically related to the question.

Third, the FAISS based retrieval mechanism searches the indexed knowledge base and identifies relevant contextual information. The retrieved content is supplied to the language model together with the user's question.

Finally, the language model generates a natural-language response using the supplied context. Where necessary, the response is translated into the language expected by the user and returned through the conversational interface.

The purpose of this process is to connect flexible natural-language interaction with domain-specific information retrieval. Rather than treating the chatbot as an independent source of financial decisions, the component is intended to provide information that helps users understand relevant services and procedures.

The NLP component is therefore an information-access mechanism within SmartGrama. It does not independently determine final welfare eligibility, approve loans, or replace the decision-making responsibilities of authorized institutions.

1.3 Problem Statement

Citizens who require welfare assistance or micro-loan services may experience difficulties obtaining accurate and understandable information because relevant details are distributed across different sources and expressed using formal or unfamiliar terminology. Conventional digital interfaces may require users to navigate multiple sections or understand predefined categories before locating the information they need.

Language differences introduce an additional challenge. A system designed primarily for English-language interaction may be less accessible to users who prefer Sinhala. Even when information is available electronically, the absence of an effective multilingual conversational interface can prevent users from obtaining the required guidance efficiently.

Existing rule-based chatbots can automate frequently asked questions, but their performance depends heavily on predefined rules, keywords, and response patterns. Consequently, a chatbot may fail to recognize two questions as equivalent when they are expressed using different wording.

General-purpose Large Language Models provide more flexible conversational interaction. However, they may generate responses that are unsupported by the specific information required in welfare and microfinance contexts. Fluent language generation alone does not guarantee factual correctness, especially when questions concern institutional procedures, eligibility requirements, or service-specific conditions.

Keyword-based information retrieval also presents limitations because users may describe a concept differently from the wording used in the source documents. A question about obtaining financial assistance, for example, may use terminology that differs from the language appearing in an official document.

The main problem addressed by this research is therefore the lack of an integrated, multilingual, domain-specific conversational mechanism that can interpret natural-language questions, retrieve relevant welfare and micro-loan information, and generate understandable responses grounded in that information.

To address this problem, the research proposes a multilingual RAG-based NLP component that combines semantic retrieval, language processing, an external knowledge base, and LLM-based response generation.

The effectiveness of the proposed approach must be established through systematic evaluation of retrieval relevance, response correctness, contextual grounding, and multilingual response quality. The successful execution of the application alone is not sufficient to demonstrate that it consistently provides accurate information.

1.4 Research Gap

Previous research has demonstrated the potential of conversational AI, semantic language representations, and retrieval-augmented language generation. Large Language Models can generate natural-language responses, while sentence-embedding techniques enable semantically similar texts to be compared without requiring exact keyword matches. RAG combines these capabilities by supplying retrieved information to a language model during response generation.

Nevertheless, several challenges remain relevant to the SmartGrama application domain.

First, general-purpose conversational systems may not have direct access to the current, institution-specific information required to answer questions about welfare and micro-loan services. Their responses can therefore contain unsupported information when the required facts are not available in the model's pretrained knowledge.

Second, keyword-dependent retrieval may not adequately handle different expressions of the same information need. Users may formulate questions informally or use terminology that differs from the wording found in official documents.

Third, multilingual interaction introduces additional processing requirements. Sinhala queries may need translation before retrieval, and translation errors can influence the information retrieved and the response generated. Consequently, multilingual support cannot be evaluated solely by confirming that the system accepts Sinhala input.

Fourth, the quality of a domain-specific assistant depends on the completeness, reliability, and maintenance of its knowledge base. If relevant information is missing or outdated, semantic retrieval cannot independently guarantee a correct answer.

The research gap addressed by this thesis is the need for a practical conversational framework that integrates semantic retrieval, domain-specific knowledge grounding, LLM-based generation, and Sinhala-English interaction for welfare and micro-loan information access within the Sri Lankan context.

The proposed component investigates this integration through a modular architecture using Sentence Transformers, FAISS, translation functionality, Flask, and Ollama.

The research contribution is the application and evaluation of these established techniques within the SmartGrama domain. It is not the invention of a new language model or a new RAG algorithm. Instead, the work focuses on integrating the technologies into a functional component and examining their suitability for multilingual, domain-specific information assistance.

1.5 Aim and Objectives

1.5.1 Main Aim

The main aim of this research is to design, implement, and evaluate a multilingual Retrieval-Augmented Generation-based Natural Language Processing component that improves access to welfare and micro-loan information through natural-language interaction in English and Sinhala.

1.5.2 Specific Objectives

The research is guided by the following objectives.

Objective 1: To identify information-access challenges associated with welfare and micro-loan services.

This objective examines the difficulties users may experience when locating relevant information, understanding administrative terminology, and interacting with conventional digital information systems.

Objective 2: To develop a domain specific knowledge base.

This objective focuses on organizing relevant welfare and micro-loan information into searchable resources that can support context-aware conversational responses.

Objective 3: To implement semantic information retrieval.

This objective involves converting knowledge-base content and user queries into vector representations and identifying relevant content through similarity-based retrieval.

Objective 4: To integrate retrieval with language-model response generation.

This objective implements the RAG workflow, in which retrieved information is supplied to an LLM together with the user's question to support domain-grounded responses.

Objective 5: To provide English and Sinhala conversational interaction.

This objective incorporates language processing and translation functionality to enable users to submit questions and receive responses in the supported languages.

Objective 6: To evaluate the performance of the NLP component.

This objective focuses on assessing retrieval relevance, response correctness, relevance, groundedness, completeness, and multilingual response quality using an appropriate evaluation dataset and documented assessment procedures.

The extent to which each objective has been achieved will be determined using implementation evidence and evaluation results presented in the later chapters.

1.6 Research Questions

The research is guided by the following questions.

RQ1: How effectively can semantic retrieval identify relevant welfare and micro loan information when users express their questions using different wording?

RQ2: To what extent does integrating retrieved domain-specific information into language-model response generation improve the factual grounding of conversational responses?

RQ3: What challenges arise when supporting English and Sinhala interaction through a translation based NLP pipeline?

RQ4: How does the quality of the knowledge base influence the relevance, correctness, and completeness of generated responses?

RQ5: What technical and evaluation limitations affect the suitability of the proposed NLP component for deployment within the SmartGrama platform?

These questions establish the relationship between the identified problem, the proposed architecture, and the evaluation activities described in this thesis.

1.7 Scope, Limitations, and Assumptions

1.7.1 Scope of the Research

The scope of this research is limited to the NLP component responsible for conversational information access within the SmartGrama platform.

The component focuses on the following areas:

- Natural language query processing.
- English and Sinhala conversational interaction.
- Translation based language processing.
- Domain specific knowledge base preparation.
- Semantic embedding generation using Sentence Transformers.
- Similarity based retrieval using FAISS.
- Contextual response generation through an LLM accessed using Ollama.
- Integration of the NLP service with a React based conversational interface.
- Evaluation of retrieval and response quality.

The knowledge domain includes welfare information, micro-loan information, application procedures, eligibility related guidance, required documentation, repayment-related information, and other relevant service information available in the knowledge base.

The thesis examines the NLP component's contribution to the overall platform rather than attempting to evaluate every subsystem of SmartGrama independently.

1.7.2 Limitations

Several limitations must be considered when interpreting the results.

First, translation based Sinhala processing can introduce errors that affect semantic retrieval and response generation.

Second, retrieval quality depends on the content, organization, and completeness of the knowledge base.

Third, response quality can vary according to the language model selected for local execution and the computational resources available.

Fourth, a technically operational prototype does not necessarily demonstrate reliable performance across all question types. Formal testing is required to assess its effectiveness.

Fifth, the research is focused on information assistance. The component is not intended to make final decisions concerning loan approval, welfare eligibility, or the allocation of financial benefits.

Finally, the findings may not generalize to other languages, service domains, or institutional environments without further investigation.

1.7.3 Assumptions

The research adopts the following assumptions.

1. The knowledge base contains information relevant to the questions used during evaluation.
2. The source information is sufficiently accurate for its intended purpose and can be maintained when policies or service procedures change.
3. The embedding model can produce representations that are useful for retrieving relevant knowledge base segments.
4. The translation mechanism can support the intended English and Sinhala interaction pathways, although its output may contain errors.
5. The selected language model can process the retrieved context and generate responses within the available computational environment.
6. The evaluation dataset represents the categories of questions that the component is intended to support.

1.8 Individual Contribution

The individual contribution of this research is the design, implementation, and evaluation of the NLP based conversational information access component within the SmartGrama platform.

The component combines semantic retrieval and LLM based response generation to support natural language questions about welfare and micro loan services. Its principal contribution is the integration of these capabilities with a translation based English and Sinhala interaction pathway.

The work covers the organization of domain specific knowledge, processing of user queries, generation of semantic embeddings, similarity based retrieval, preparation of contextual information, and interaction with the language model. It also considers how the resulting responses are returned through the conversational interface.

The research examines the relationship between retrieved context and generated responses rather than evaluating the chatbot solely according to its ability to produce fluent language. This distinction is important because the intended application requires information that is relevant to the domain and supported by available knowledge.

The individual component is also designed to operate as part of a larger application architecture. The React frontend provides the user interaction layer, while the Flask backend coordinates the NLP workflow and communication with the retrieval and generation components.

The following contributions are addressed in the thesis:

1. Development of a domain specific conversational information access workflow.
2. Integration of sentence embeddings and FAISS based semantic retrieval.
3. Integration of retrieved contextual information with a locally deployed LLM.
4. Support for English and Sinhala interaction through translation functionality.
5. Definition of an evaluation approach for retrieval relevance, response quality, and contextual grounding.
6. Identification of limitations and future improvements relevant to the deployment of multilingual conversational assistance.

1.9 Thesis Structure

This thesis is organized into nine chapters, following the individual thesis template.

2.LITERATURE REVIEW AND RELATED WORK

2.1 Review Method and Scope

The literature review examines the theoretical foundations and existing research relevant to developing a multilingual Natural Language Processing component for welfare and micro loan information access. The review focuses on conversational artificial intelligence, Large Language Models, Retrieval Augmented Generation, semantic sentence embeddings, vector based information retrieval, and multilingual language processing.

The purpose of the review is to establish the theoretical justification for the proposed SmartGrama NLP component and identify limitations that the research aims to address. Particular attention is given to the relationship between semantic retrieval and language generation, the importance of domain-specific knowledge, and the challenges of supporting Sinhala and English interaction.

2.1.1 Literature Review Approach

The review is organized thematically rather than presenting each publication as an isolated study. This approach makes it possible to compare related techniques, identify their strengths and limitations, and explain how they contribute to the proposed architecture.

The principal themes examined are:

1. Natural Language Processing and conversational AI.
2. Large Language Models and natural language response generation.
3. Retrieval Augmented Generation for domains pecific question answering.
4. Semantic sentence embeddings and similarity based retrieval.
5. Vector search and the role of FAISS.
6. Multilingual NLP and Sinhala language processing.
7. Conversational assistants for welfare and financial information.
8. Evaluation methods for retrieval and generated responses.

The main academic sources are those cited in the research paper and project proposal. These include Brown et al. on large language models, Lewis et al. on Retrieval Augmented Generation, Devlin et al. on BERT, Reimers and Gurevych on sentence embeddings, and Ranasinghe et al. on Sinhala language models and evaluation.

The review also considers the technologies used in the proposed implementation, including Sentence Transformers, FAISS, Flask, React, translation functionality, and Ollama.

The reviewed literature provides the conceptual foundation for the component. However, the existence of a particular technique does not automatically demonstrate its effectiveness in the SmartGrama context. The suitability of the integrated solution must be established through implementation and evaluation.

2.1.2 Scope of the Review

The review is limited to research and technologies that directly influence the design of the individual NLP component. General blockchain implementation, detailed credit scoring

algorithms, and the internal operation of unrelated SmartGrama components are outside the primary scope of this chapter.

Nevertheless, the relationship between the NLP component and the wider platform is considered where it affects the reliability, accessibility, and delivery of information.

The literature review emphasizes four requirements for the proposed system: users should be able to express questions naturally, the system should retrieve relevant domain specific information, generated responses should be grounded in available evidence, and the interaction mechanism should support the intended languages.

2.2 Domain and Theoretical Foundations

2.2.1 Natural Language Processing

Natural Language Processing is a field of artificial intelligence concerned with enabling computers to process, interpret, and generate human language. It combines computational methods, linguistic representations, and statistical or machine learning techniques to perform tasks such as text classification, information extraction, machine translation, semantic analysis, and question answering.

NLP has become an important foundation for conversational applications because it allows users to communicate with software using ordinary language instead of relying exclusively on structured commands. Jurafsky and Martin provide a comprehensive treatment of the linguistic and computational principles underlying modern language processing systems [9].

Within SmartGrama, NLP is used to support the interpretation of questions concerning welfare and micro-loan information. Users may ask about application requirements, eligibility conditions, repayment procedures, or the documents needed to access a service. These questions may differ in wording even when they seek similar information.

For example, the following questions may express related information needs:

- What documents are required to apply for a loan?
- Which documents should I submit with my loan application?
- What do I need to prepare before applying for a micro loan?

A system that relies only on exact keywords may not consistently recognize these expressions as equivalent. Semantic representations provide an alternative by representing text according to its learned linguistic meaning.

In the proposed component, NLP is therefore closely connected to semantic retrieval and contextual response generation. Its purpose is not limited to identifying individual words; it also supports the process of connecting a user's question with relevant information in the knowledge base.

2.2.2 Conversational Artificial Intelligence

Conversational artificial intelligence refers to systems that enable interaction between people and software through natural language exchanges. Such systems can be implemented using predefined rules, intent classification models, retrieval mechanisms, generative language models, or combinations of these techniques.

Earlier conversational systems commonly depended on explicitly defined patterns and response rules. These systems can be effective when the number of possible interactions is limited and the required responses are predictable. However, they require substantial rule maintenance when users express similar questions in many different ways.

Neural conversational models introduced more flexible approaches to response generation. Vinyals and Le demonstrated the potential of neural networks for conversational response generation. Their work contributed to the broader development of data driven conversational systems.

More recent conversational systems use pretrained language models to interpret questions and generate responses. These systems can provide a more flexible interface because users are not necessarily restricted to predefined questions.

Nevertheless, conversational fluency and factual reliability are different properties. A response may be grammatically correct and appear convincing while containing information that is not supported by an authoritative source.

This distinction is particularly important for welfare and micro loan information. Users may rely on the answers to understand administrative requirements or prepare for a financial application. The conversational system should therefore prioritize relevance, factual grounding, and clarity rather than conversational fluency alone.

The proposed SmartGrama component addresses this requirement by combining conversational generation with an external knowledge retrieval mechanism.

2.2.3 Large Language Models

Large Language Models are neural language models trained on large collections of text to learn linguistic patterns and generate contextually appropriate sequences. Their capabilities can include text generation, summarization, translation, question answering, and conversational interaction.

Brown et al. demonstrated that scaling language models can support a range of tasks through prompting and few shot learning [1]. Devlin et al. introduced BERT, a pretrained bidirectional language representation model that established an important foundation for subsequent NLP applications [4].

These developments illustrate two important directions in language technology: learning general purpose linguistic representations and using pretrained models to support multiple downstream tasks.

For the SmartGrama NLP component, a language model is used to produce a readable response from the user's question and the retrieved information. The model is not intended to serve as the sole authority on welfare policies or micro-loan requirements.

A major limitation of general purpose language models is that their responses are influenced by information encoded during training and by the context supplied at inference time. They may not have access to the latest institutional procedures or the precise information contained in a particular knowledge base.

Furthermore, a model may generate unsupported statements when the supplied context is insufficient. This is commonly described as hallucination in generative AI.

Consequently, using an LLM without a domain specific retrieval mechanism may be inadequate for applications that require information tied to particular documents or institutional procedures.

The proposed architecture addresses this limitation by retrieving relevant domain information before generating a response. The retrieved material provides additional context that the language model can use when constructing its answer.

2.2.4 Retrieval Augmented Generation

Retrieval Augmented Generation is an approach that combines information retrieval with generative language modelling. Lewis et al. introduced RAG for knowledge intensive NLP tasks by combining pretrained language models with an external non parametric knowledge source [3].

In a conventional generative approach, a user submits a question directly to the language model. The model then produces an answer using the information represented in its parameters and the current conversation context.

In a RAG based approach, the system first searches an external knowledge source for information relevant to the question. The retrieved content is then provided to the language model as additional context.

The general RAG workflow is:

User Query → Information Retrieval → Retrieved Context → Language Model → Generated Response

This separation between retrieval and generation provides several benefits. The knowledge base can be updated independently of the language model, relevant source information can be supplied during response generation, and the system can be evaluated at both the retrieval and generation stages.

However, RAG does not automatically guarantee correctness. If the retrieval stage selects irrelevant information, if the source material is outdated, or if the language model misinterprets the retrieved context, the resulting response may still be inaccurate.

Gao et al. discuss RAG architectures and their associated challenges in their survey of retrieval augmented language models [5]. Their work provides a broader basis for understanding the design choices and evaluation requirements of such systems.

For SmartGrama, RAG is suitable because the knowledge domain includes specific welfare and micro-loan information that may be documented separately from the language model.

The external knowledge base provides a mechanism for supplying that information when a user asks a question.

The proposed component therefore follows the principle that the language model should generate responses using retrieved domain information rather than relying exclusively on pretrained knowledge.

2.2.5 Semantic Sentence Embeddings

Semantic sentence embeddings represent sentences or text segments as numerical vectors. These vectors are generated by an embedding model and can be compared to estimate semantic similarity.

A central advantage of semantic embeddings is that texts with related meanings may have similar vector representations even when they use different words.

Reimers and Gurevych introduced Sentence BERT, an approach for producing semantically meaningful sentence embeddings that can be compared efficiently [6]. This work established an important foundation for sentence level semantic search.

Within the SmartGrama component, Sentence Transformers are used to create vector representations of knowledge base content and user queries. The vectors enable the retrieval system to search for relevant text segments based on their semantic representations.

For example, a question about the documents needed for a loan application may be compared with a knowledge base segment describing application documentation, even when the wording is not identical.

The process consists of three main operations:

1. Convert knowledge base text segments into embeddings.
2. Convert the processed user question into an embedding using the appropriate model.
3. Compare the query representation with indexed document representations to identify relevant content.

The effectiveness of this process depends on the embedding model, the language used, the content of the knowledge base, and the similarity-search configuration.

An important consideration is language compatibility. An embedding model that performs well for English does not necessarily provide equally effective representations for Sinhala. The chosen model and the translation pathway must therefore be evaluated in the intended language processing environment.

2.2.6 Vector Similarity Search and FAISS

Vector similarity search identifies items whose vector representations are similar to a given query vector. It is widely used in semantic search, recommendation systems, and retrieval-augmented applications.

FAISS, developed by Johnson, Douze, and Jégou, provides tools for efficient similarity search and clustering of dense vectors [7]. It supports different indexing strategies and enables retrieval over vector collections without requiring every comparison to be implemented manually.

In the proposed NLP component, FAISS indexes the embeddings generated from the domain specific knowledge base. When a user submits a question, the query is converted into a vector and searched against the indexed representations.

The retrieval subsystem then returns the most relevant candidate segments according to the configured similarity search method.

The quality of the retrieved results depends on several factors, including the embedding model, text segmentation, vector normalization where applicable, the similarity metric, and the number of results requested.

FAISS is a retrieval tool rather than a language model or a complete knowledge-management system. It identifies vector matches, but it does not independently establish whether the underlying information is authoritative or whether the generated answer is correct.

For this reason, FAISS is used as one stage of the larger RAG workflow. Its output must be combined with suitable knowledge sources and response generation controls.

2.2.7 Multilingual Natural Language Processing

Multilingual NLP aims to support language-processing tasks across more than one language. It is relevant to applications in which users communicate using different linguistic forms but require access to the same underlying information.

Multilingual systems can use several approaches, including translation based processing, multilingual embedding models, and language models trained on multiple languages.

Reimers and Gurevych investigated multilingual sentence embeddings through knowledge distillation, demonstrating an approach for making sentence representations useful across languages [8].

Translation-based systems provide another practical method. A query written in one language can be translated into a retrieval compatible language, processed by the retrieval system, and answered using the available knowledge. The resulting response can then be translated into the user's preferred language.

The SmartGrama NLP component adopts a translation based approach to support English and Sinhala interaction. According to the research paper, translation functionality is incorporated using the deep translator library and Google Translator functionality.

This approach avoids the need to train a new Sinhala specific conversational model as part of the component. However, it introduces an additional processing stage that can affect the accuracy of the final response.

Translation errors may change important terminology, omit details, or alter the meaning of a question. In a welfare or micro loan context, such errors could affect the interpretation of requirements or procedures.

Therefore, the multilingual capability must be evaluated separately from the English-language retrieval and response generation process. A system that accepts Sinhala text has not necessarily demonstrated that it understands the intended meaning correctly.

2.2.8 Sinhala Language Processing

Sinhala language processing presents specific challenges related to linguistic structure, language resources, and evaluation. Ranasinghe et al. examine Sinhala encoder only language models and evaluation Their research highlights the importance of language-specific resources and systematic evaluation for Sinhala NLP.

The availability of suitable datasets, pretrained models, and evaluation benchmarks influences the development of Sinhala language processing applications. These factors are important when designing systems that need to preserve the meaning of user questions and generate understandable responses.

For the SmartGrama component, the selected approach uses translation functionality to connect Sinhala interaction with the existing retrieval and generation pipeline.

This design offers a practical integration path, but it does not remove the need to evaluate Sinhala-specific performance. The system should be tested for translation correctness, retrieval relevance after translation, preservation of domain terminology, and the clarity of the final Sinhala response.

Future development could investigate multilingual embedding models that directly support Sinhala, Sinhala specific language models, or hybrid retrieval approaches. Such alternatives should be compared experimentally rather than assumed to outperform the existing implementation.

2.2.9 Knowledge Bases and Domain Specific Information

A knowledge base is a structured or semi-structured collection of information that a system can use to answer questions or support decision-making. In a RAG architecture, the knowledge base provides the external information retrieved during question answering.

For SmartGrama, the knowledge domain includes welfare service descriptions, eligibility-related information, application procedures, required documents, micro-loan information, repayment-related information, and frequently asked questions.

The quality of the knowledge base directly influences the information available to the retrieval subsystem. If a relevant policy or procedure is missing, the system may not be able to retrieve the evidence required to answer a question accurately.

Knowledge-base preparation therefore involves more than storing documents. Source material must be selected, processed, divided into meaningful text segments, and indexed for

retrieval. The source information must also be maintained when institutional requirements change.

For an application involving financial or welfare information, the authority and currency of the source material are particularly important. A retrieved passage may be semantically relevant but still be unsuitable if it is outdated or does not apply to the user's intended service.

The proposed component uses domain-specific knowledge to ground the generated response. However, its reliability remains dependent on the quality and completeness of the underlying information.

2.2.10 Theoretical Basis of the Proposed Component

The proposed SmartGrama NLP component combines the preceding concepts into a single conversational information access workflow.

Natural Language Processing provides the general framework for processing user questions. Semantic embeddings represent the query and knowledge base segments in vector form. FAISS performs similarity based retrieval, and RAG supplies the retrieved information to the language model for response generation.

Translation functionality supports interaction in English and Sinhala, while the React frontend and Flask backend provide the interface and application level coordination.

This conceptual framework illustrates the relationship between the principal technologies. It does not imply that every stage has been experimentally proven to improve performance.

2.3 Related Research

2.3.1 Research on Large Language Models

Brown et al. investigated the capabilities of large language models through few shot learning [1]. Their work demonstrated that a sufficiently capable pretrained language model can perform a variety of tasks using instructions and examples supplied through its input context.

This research is relevant because modern conversational assistants can use a general-purpose language model to generate responses without developing a separate model for every application domain.

However, general language capabilities do not guarantee that a model will possess the exact information needed for a particular welfare or micro loan question. The SmartGrama component therefore uses the LLM as a response generation mechanism supported by external information retrieval.

2.3.2 Research on Retrieval-Augmented Generation

Lewis et al. introduced Retrieval Augmented Generation for knowledge intensive NLP tasks [3]. Their approach combines parametric knowledge stored within a pretrained model with information available through an external non-parametric memory.

The work provides the principal theoretical justification for retrieving domain-specific information before generating a response. It also demonstrates why information retrieval and language generation can be treated as complementary parts of a question-answering system.

Gao et al. subsequently surveyed RAG techniques and discussed the broader development of retrieval augmented language models [5]. This literature is relevant to the SmartGrama component because it identifies retrieval quality, contextual use, and system design as important considerations.

The proposed implementation applies the general RAG principle using Sentence Transformers, FAISS, and Ollama. It does not reproduce every architecture described in the literature; instead, it adopts a modular retrieval-and-generation workflow suited to the project's requirements.

2.3.3 Research on Sentence Embeddings

Reimers and Gurevych introduced Sentence BERT to produce sentence representations suitable for semantic comparison [6].

Their work is directly relevant to the retrieval stage of the proposed component. A semantic retrieval mechanism must represent the meaning of a question and compare it with the content of candidate knowledge base segments.

The SmartGrama implementation uses Sentence Transformers for this purpose. The embeddings provide a common representation that allows the retrieval mechanism to search for semantically related content.

Nevertheless, the performance of the implemented system cannot be inferred solely from the reported capabilities of Sentence BERT. The selected embedding model, the characteristics of the knowledge base, and the nature of the test queries can influence the actual retrieval results.

2.3.4 Research on Efficient Vector Search

Johnson, Douze, and Jégou presented research on efficient similarity search over large vector collections, associated with the FAISS system [7].

Their work is relevant because vector-based retrieval becomes an important technical requirement when a system represents documents as dense numerical embeddings.

The SmartGrama component uses FAISS to index and search the generated vectors. This supports the retrieval architecture by separating the similarity-search operation from the language-generation process.

However, efficient vector search and relevant information retrieval are not identical. A system may quickly return the nearest vectors while still retrieving content that does not answer the user's question. Retrieval relevance must therefore be evaluated independently from execution speed.

2.3.5 Research on Multilingual Sentence Embeddings

Reimers and Gurevych investigated methods for creating multilingual sentence embeddings using knowledge distillation [8].

This research is relevant because multilingual semantic retrieval can reduce dependence on translating every query into a single retrieval language. When an embedding model produces comparable representations across languages, it may be possible to retrieve relevant content directly from multilingual queries.

The current SmartGrama approach instead incorporates translation functionality into its processing pipeline. This represents a practical design choice, but it also introduces the possibility that translation errors will affect retrieval.

A meaningful comparison between translation-based retrieval and direct multilingual retrieval would require an appropriate evaluation dataset and consistent test conditions. Such a comparison is identified as a potential direction for further research.

2.3.6 Research on Sinhala NLP

Ranasinghe et al. investigated Sinhala encoder-only language models and evaluation [10]. Their work is relevant to the development of language technologies for Sinhala and to the need for evaluation methods that reflect the characteristics of the language.

The SmartGrama component does not claim to introduce a new Sinhala language model. Instead, it uses translation based processing to connect Sinhala questions with the existing semantic retrieval and generation pipeline.

The relevant research therefore supports the need to evaluate Sinhala performance independently. Particular attention should be paid to preserving the intended meaning of questions, retrieving appropriate information, and producing understandable responses.

2.3.7 Research on Financial Information Access and Conversational Assistance

The project proposal identifies access to financial information as an important concern in the context of microfinance in Sri Lanka. It discusses the role of microfinance institutions and the difficulties that users may experience when obtaining information about financial services.

The proposal also reviews neural conversational models, multilingual chatbot research, conversational frameworks, and Sinhala language processing work.

These sources establish the relevance of conversational interfaces for improving access to information. However, the literature and project proposal do not, by themselves, demonstrate that a particular chatbot architecture will be accurate enough for real-world welfare or financial guidance.

The SmartGrama component addresses this concern by combining a conversational interface with domain specific retrieval. The system is intended to help users locate and understand relevant information rather than independently approve loans or determine eligibility for welfare benefits.

Research	Main contribution	Relevance to SmartGrama	Limitation for the current application
Brown et al. [1]	Few-shot capabilities of large language models	Supports the use of pretrained LLMs for response generation	Does not establish correctness for SmartGrama-specific information
Lewis et al. [3]	Retrieval-Augmented Generation	Provides the foundation for retrieval-grounded generation	RAG still depends on relevant retrieval and suitable source material
Devlin et al. [4]	BERT language representations	Provides background on pretrained language representations	Does not directly validate the selected embedding model
Gao et al. [5]	Survey of RAG architectures	Provides broader context for retrieval-augmented system design	Does not establish the performance of this particular implementation
Reimers and Gurevych [6]	Sentence-BERT sentence embeddings	Supports semantic representation and similarity comparison	Actual retrieval performance depends on the selected model and dataset
Johnson et al. [7]	Efficient vector similarity search	Supports the use of FAISS for vector retrieval	Efficient search does not guarantee semantic relevance
Reimers and Gurevych [8]	Multilingual sentence embeddings	Provides an alternative direction for multilingual retrieval	Direct multilingual retrieval is not established by the current implementation

Table 2 Summary of Related Research on Language Models, Retrieval-Augmented Generation, and Semantic Search for SmartGrama

2.3.8 Summary of Related Research

The reviewed research provides complementary foundations for the proposed component.

Large Language Models support flexible natural language generation. RAG provides a method for integrating external knowledge with generation. Sentence embeddings support semantic comparison, and FAISS provides an efficient mechanism for vector similarity search. Multilingual language processing research establishes alternative approaches to

Cross language interaction, while Sinhala NLP research highlights the need for language specific evaluation.

The contribution of the SmartGrama component lies in integrating these established approaches into a domain specific conversational information access workflow. The research must then determine whether this integration is effective for the intended welfare and micro loan questions.

2.4 Existing Systems, Products, and Techniques

2.4.1 Rule Based Chatbots

Rule based chatbots generate responses using predefined patterns, keywords, decision trees, or explicitly configured conversational rules.

These systems are useful for predictable interactions, such as providing office hours, explaining a fixed application procedure, or directing users to a particular service.

Their principal advantage is that the responses can be controlled and inspected. If a question matches a predefined pattern, the system can return an established answer without relying on generative language modelling.

However, rule based systems require developers to anticipate the ways in which users may express their questions. They may fail when the same information need is expressed through an unfamiliar phrase or a different language.

For example, questions concerning loan documentation may use several alternative expressions. A rule based chatbot may require additional patterns for each variation.

The SmartGrama component addresses this limitation through semantic retrieval and LLM based response generation. This provides a more flexible method of matching user questions to relevant domain information, although it introduces additional complexity and evaluation requirements.

2.4.2 General Purpose LLM Chatbots

General purpose LLM chatbots use pretrained language models to interpret prompts and generate responses. They can handle a wide variety of topics without requiring a separate predefined answer for every possible question.

Their flexibility makes them suitable for natural language interfaces. However, their responses may not reflect the exact content of an institutional policy, a particular loan procedure, or a current welfare requirement.

A general purpose chatbot may generate a plausible answer even when the required information is unavailable. This creates a risk when users interpret the generated content as authoritative.

The SmartGrama architecture differs by retrieving information from a domain specific knowledge base before generating a response. The intended benefit is improved grounding in the information available to the application.

This distinction does not mean that every RAG response will be more accurate than every response from a general purpose LLM. A fair comparison requires testing both approaches using the same questions and an appropriate reference standard.

2.4.3 Keyword Based Information Retrieval

Keyword based retrieval identifies documents or passages containing terms that match the user's query. It is straightforward to implement and can be effective when users employ terminology similar to that found in the source documents.

However, exact word overlap is an incomplete measure of semantic relevance. A user may describe a requirement using ordinary language, while the source document uses formal administrative terminology.

Semantic retrieval addresses this limitation by comparing vector representations rather than relying exclusively on exact keyword matches.

The SmartGrama component uses Sentence Transformers and FAISS to support this form of retrieval. The objective is to identify relevant information even when the query and source passage do not share identical wording.

Nevertheless, semantic retrieval may also return incorrect or loosely related passages. It should therefore be assessed using a labelled evaluation dataset rather than assumed to be reliable in all cases.

2.4.4 Retrieval Augmented Conversational Systems

RAG based conversational systems combine information retrieval with natural-language generation. They are particularly relevant when the system must answer questions using a defined collection of documents.

The approach offers a practical way to update domain information independently of the language model. If a policy document changes, the knowledge base and vector index can be updated without necessarily retraining the entire LLM.

The main technical challenge is ensuring that the retrieved content is relevant and that the language model uses it correctly.

The SmartGrama component adopts this architecture to provide information about welfare and micro loan services. The system retrieves domain specific passages and uses them as contextual input to the LLM.

2.4.5 Comparison of Existing Approaches

The principal approaches can be compared according to their flexibility, reliance on predefined information, retrieval capabilities, and relevance to the SmartGrama requirements.

Criterion	Rule based chatbot	General purpose LLM	Keyword retrieval	RAG based conversational system
Natural-language flexibility	Limited by configured rules	Generally high	Limited to search behaviour	Combines retrieval with generation
External knowledge retrieval	Only if explicitly implemented	Not necessarily available	Yes	Yes
Domain-specific grounding	Depends on predefined responses	Not guaranteed	Returns source text but does not generate answers by itself	Uses retrieved information as generation context
Handling paraphrases	Requires rules or patterns	Often capable	Depends on matching method	Can use semantic retrieval
Response generation	Predefined responses	Generated responses	Usually returns documents or passages	Generated responses based on retrieved context
Knowledge updates	Rules or responses must be updated	Depends on model and external tools	Index or documents can be updated	Knowledge base and index can be updated
Principal limitation	Limited flexibility	May generate unsupported claims	May miss semantically related content	Depends on retrieval, source quality, and generation behaviour

Table 3 Comparison of Conversational and Information Retrieval Approaches

The comparison shows why RAG is a suitable candidate for the SmartGrama application. It combines the flexibility of natural language generation with the ability to retrieve information from a defined knowledge source.

However, the table presents general characteristics of the approaches, not measured results from a controlled experiment. A numerical performance comparison would require a common dataset and a consistent evaluation procedure.

2.4.6 Comparison of Relevant Technologies

The technologies used in the proposed implementation perform different functions and should not be treated as interchangeable.

The technologies form a modular pipeline. React is responsible for interaction, Flask coordinates application logic, Sentence Transformers generates embeddings, FAISS performs similarity search, and Ollama provides access to the language model.

The translation library supports the multilingual pathway, while the knowledge base supplies the domain information needed for the intended application.

The combination provides a practical implementation architecture. Nevertheless, each stage introduces potential failure points, so testing must cover the complete pipeline as well as its individual components.

Technology	Primary function	Contribution to the component	Important consideration
React	Frontend development	Provides the conversational user interface	Must communicate correctly with the backend
Flask	Backend application framework	Coordinates incoming requests and NLP processing	Requires appropriate error handling and request validation
Sentence Transformers	Sentence embedding generation	Represents queries and knowledge base text as vectors	Performance depends on the selected model and language support
FAISS	Vector similarity search	Retrieves candidate text segments from the indexed embeddings	Retrieval quality depends on the index, embeddings, and search configuration
deep-translator	Translation integration	Supports the translation-based language-processing pathway	Translation quality and external service availability may affect performance
Ollama	Local LLM execution interface	Provides access to the selected locally deployed language model	Response quality and latency depend on the model and hardware
Domain-specific knowledge base	Stores relevant information	Supplies contextual evidence for response generation	Must be accurate, sufficiently complete, and maintained

Table 4 Technologies Used in the Proposed NLP Architecture

2.4.7 Suitability of the Proposed Architecture

The proposed architecture is suitable for investigation because it separates information retrieval from response generation and keeps the knowledge source independent of the language model.

This separation supports modular development and makes it possible to evaluate retrieval quality independently from response quality. It also allows the knowledge base to be maintained without necessarily changing the frontend or retraining the LLM.

The local execution of the language model through Ollama can reduce dependence on a commercial hosted LLM API. However, local execution does not automatically guarantee privacy, security, or low operating costs. These properties depend on the wider implementation, data handling practices, and deployment environment.

Similarly, translation based interaction offers a practical way to support Sinhala without training a new model, but it introduces a dependency on translation quality and service availability.

The architecture should therefore be regarded as a design choice whose effectiveness must be evaluated, rather than as a guarantee of superior performance.

2.5 Gap Synthesis and Conceptual Framework

2.5.1 Synthesis of the Identified Research Gap

The literature review establishes that conversational AI, semantic retrieval, and retrieval-augmented generation offer complementary capabilities.

Rule based systems can provide controlled responses but may struggle with variations in user wording. General purpose LLMs provide flexible interaction but may produce information that is unsupported by the intended knowledge source. Keyword based retrieval can locate relevant text but may fail when the user's terminology differs from that used in the documents.

Semantic embedding methods offer a way to compare text according to learned meaning, while vector search provides an efficient mechanism for finding similar representations. RAG connects these retrieval capabilities with generative response construction.

Multilingual interaction introduces another requirement. Although translation can support communication across languages, its effect on retrieval and generation must be assessed. This is particularly important when a system is intended to support Sinhala and English.

The resulting research gap is not the absence of chatbots, embeddings, vector databases, or RAG techniques individually. Rather, it concerns the integration and evaluation of these techniques for the specific purpose of providing domain-grounded welfare and micro-loan information through a bilingual conversational interface.

The proposed SmartGrama component addresses this gap by integrating:

1. A domain specific knowledge base.
2. Translation based English and Sinhala interaction.
3. Sentence Transformer embeddings.
4. FAISS based semantic retrieval.
5. LLM based response generation through Ollama.
6. A React frontend and Flask backend.
7. An evaluation framework covering retrieval and response quality.

The research focuses on whether this integrated workflow can provide relevant, understandable, and appropriately grounded answers to representative user questions.

2.5.2 Research Gap Matrix

The proposed responses identify design decisions and evaluation priorities. They should not be interpreted as proof that the identified limitations have already been eliminated.

Identified limitation	Consequence for the application	Proposed response
Dependence on predefined chatbot patterns	Users may struggle when questions are expressed differently	Use semantic retrieval and generative response construction
Lack of domain-specific context in a general LLM	Answers may not reflect the required source information	Retrieve information from a dedicated knowledge base
Dependence on exact keyword matching	Relevant passages may be missed when terminology differs	Use sentence embeddings and vector similarity search
Language-processing challenges for Sinhala	User intent or technical terminology may be altered	Incorporate translation and evaluate Sinhala responses separately
Uncertain response grounding	Fluent answers may contain unsupported information	Evaluate whether generated claims are supported by retrieved context
Dependence on knowledge base quality	Missing or outdated information may produce incomplete answers	Review source coverage and maintain the knowledge base

Table 5 Identified Research Gaps and the Proposed Response

2.5.3 Conceptual Framework

The conceptual framework of the research connects the quality of user input, the retrieval process, the available knowledge, and the response-generation mechanism.

The first element is the user query. The user expresses a question in English or Sinhala through the conversational interface.

The second element is language processing. Where required, the query is translated into a form suitable for the retrieval pathway. The processed query is then represented using the selected sentence embedding model.

The third element is semantic retrieval. The vector representation is compared against the indexed knowledge base vectors using FAISS. The retrieval mechanism returns candidate passages that are expected to be relevant to the user's question.

The fourth element is contextual response generation. The selected passages and the original or processed query are supplied to the language model. The model generates a response based on the information available in the supplied context.

The final element is response delivery and evaluation. The response is returned through the user interface, and its quality can be assessed according to correctness, relevance, groundness, completeness, and language quality.

The framework identifies several factors that may influence the final outcome:

- The completeness and accuracy of the knowledge base.
- The quality of the embedding model.
- The relevance of the retrieved passages.
- The quality of the translation pathway.
- The selected language model and its use of context.
- The consistency of the evaluation method.

2.5.4 Relationship Between the Research Gap and Evaluation

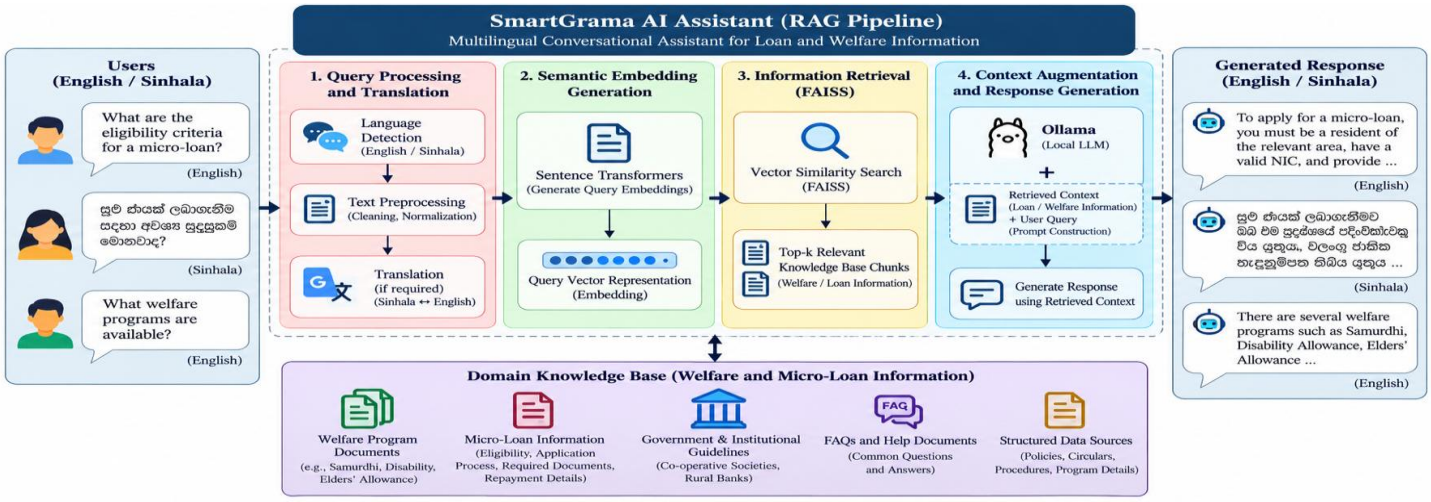


Figure 2.2: Conceptual Framework for the Multilingual RAG-Based NLP Component

Figure 1 Relationship Between the Research Gap and Evaluation

The identified research gap determines the evaluation priorities of the thesis.

If semantic retrieval is intended to improve access to relevant information, the evaluation must establish whether relevant passages are retrieved for representative questions.

If RAG is intended to improve factual grounding, generated responses must be compared with the retrieved evidence and an appropriate reference answer.

If the component is intended to support English and Sinhala, language quality and meaning preservation must be evaluated separately for each language.

If the component is intended to provide useful information about welfare and micro-loan services, the test dataset must contain representative questions from those domains, including variations in wording and questions for which the knowledge base does not contain a sufficient answer.

A comparison between RAG and a non RAG baseline can provide additional evidence regarding the contribution of retrieval. Such a comparison should use the same questions, reference information, and evaluation criteria for both configurations.

3: RESEARCH METHODOLOGY

3.1 Research Design

This research follows a design and evaluation approach to develop and assess a multilingual Natural Language Processing (NLP) component for the SmartGrama platform. The main purpose is to provide citizens with accessible conversational information about welfare programmes and micro-loan services through English and Sinhala natural-language queries. The proposed solution combines semantic information retrieval with Large Language Model (LLM) response generation through a Retrieval-Augmented Generation (RAG) architecture.

The research focuses on designing, implementing, integrating, and evaluating a working software prototype rather than training a new foundation language model. The methodology therefore concentrates on knowledge base preparation, multilingual query processing, semantic embedding generation, vector based information retrieval, contextual response generation, and systematic evaluation of the resulting answers.

The research follows a sequence of activities. First, the information access problem and the requirements of the NLP component are identified. Next, relevant SmartGrama domain information is organised into a searchable knowledge base. The information is then processed into suitable text segments, represented using semantic embeddings, and indexed for similarity based retrieval. The conversational pipeline subsequently retrieves relevant passages for a user query and supplies them to an LLM to generate a context informed answer.

The resulting prototype is evaluated using a structured dataset containing 217 question records. The evaluation examines answer correctness, retrieval relevance, response relevance, groundedness, completeness, English and Sinhala language quality, and response time. This enables the research to examine both the technical feasibility of the architecture and the limitations that may affect its reliability.

The research design is appropriate because the contribution is the practical integration of established NLP technologies into a domain specific multilingual information access system. It also provides a structured basis for identifying technical weaknesses and proposing improvements.

3.1.1 Research Approach

A predominantly quantitative and qualitative technical evaluation approach is adopted. Quantitative analysis can be applied to measurable characteristics such as response time and the proportion of answers meeting predefined evaluation criteria. Qualitative analysis is used to inspect generated responses, retrieved passages, translation effects, unsupported claims, and recurring failure patterns.

The quantitative component is based on the structured evaluation records available for the 217 questions. The qualitative component examines whether the retrieved information is

appropriate for the question and whether the generated response accurately represents the retrieved evidence.

The research does not assume that a fluent answer is necessarily correct. Instead, generated answers are assessed against expected answers and the relevant domain evidence. This distinction is important because the assistant provides information about welfare and financial services, where inaccurate guidance may mislead users.

3.1.2 Research Strategy

The research strategy consists of prototype development followed by controlled question-based evaluation. The prototype is treated as the artefact being investigated, while the evaluation dataset provides a consistent set of inputs for examining its behaviour.

The strategy includes the following activities:

1. Analyse the information needs of citizens seeking welfare and micro-loan guidance.
2. Identify and organise domain specific information for the knowledge base.
3. Preprocess and segment knowledge documents into retrievable text passages.
4. Generate semantic embeddings using Sentence Transformers.
5. Index the embeddings using FAISS for similarity based retrieval.
6. Implement English and Sinhala query processing, including translation when required.
7. Integrate retrieved context with Ollama based language generation.
8. Connect the processing pipeline to the Flask backend and React conversational interface.
9. Evaluate the system using the structured 217 question dataset.
10. Analyse response quality, system behaviour, limitations, and opportunities for improvement.

3.2 Data, Participants or Study Materials

The study primarily uses domain knowledge and a structured question based evaluation dataset. The available methodology does not rely on a statistically sampled population of human participants. Consequently, the 217 question evaluation should be described as a technical evaluation dataset rather than a survey of 217 citizens.

The principal study materials are the SmartGrama welfare and micro-loan knowledge base, the software components used to implement the RAG pipeline, and the records generated during question based evaluation.

3.2.1 Domain Knowledge Base

The knowledge base provides the external information used to support responses generated by the assistant. Its purpose is to supply domain specific evidence that may not be reliably available from the language model's internal parameters alone.

The knowledge domain includes information relevant to welfare services, micro-loan services, eligibility information, application procedures, required documentation, repayment

guidance, frequently asked questions, and other relevant SmartGrama service information where available in the source material.

Knowledge preparation involves organizing available information into a consistent format suitable for text processing and retrieval. Documents are cleaned and divided into smaller text segments so that relevant passages can be retrieved without requiring the entire knowledge collection to be included in each prompt.

The quality of the knowledge base is an important factor in the overall system's reliability. Missing, outdated, ambiguous, or contradictory information can affect both retrieval and answer generation. Therefore, the assistant should not be considered an authoritative source for a service requirement unless the corresponding information is supported by an appropriate and current source.

3.2.2 The 217 Question Evaluation Dataset

A structured evaluation dataset containing 217 question records is used to examine the behavior of the NLP component. The dataset includes questions in English and Sinhala and covers welfare, micro loan, profile, dashboard, and related information categories, as represented in the available evaluation records.

Each record provides fields that support the comparison of the system's output with an expected answer and the information retrieved during processing.

Information category	Purpose within the assistant
Welfare programme information	Provides general information about relevant welfare services and programmes.
Eligibility information	Supports questions about stated programme requirements, where such information is available.
Application procedures	Explains documented application steps and processes.
Required documentation	Supports questions about documents identified in the knowledge sources.
Micro-loan information	Provides information about micro-loan services and their documented conditions.
Repayment guidance	Supports questions about repayment procedures and related guidance available in the source material.
FAQs and service guidance	Supports common questions and general information requests.

Table 6 Categories of Information Supported by the SmartGrama NLP Assistant

The expected answer acts as a reference for evaluating response correctness and completeness. The retrieved context is used to inspect whether the information supplied to

the language model is relevant to the question and whether the final answer is supported by that information.

The dataset provides a repeatable basis for examining system behaviour. However, its size alone does not establish that the questions are statistically representative of all possible citizen queries. The conclusions must therefore remain within the scope of the available questions and evaluation procedure.

3.2.3 Software and Technical Study Materials

The software prototype is another important study material because the research investigates the behaviour of an integrated NLP pipeline.

These technologies are evaluated as parts of an integrated system rather than as independent research models. The research does not claim to have developed a new embedding architecture, vector-search algorithm, translation model, or foundation language model.

Technology	Role in the component
React	Provides the conversational user interface.
Flask	Implements the backend API and coordinates the processing workflow.
Python	Supports backend logic and NLP-related processing.
Sentence Transformers	Converts queries and knowledge segments into semantic vector representations.
FAISS	Supports similarity-based retrieval from indexed vectors.
Deep translator	Supports translation-assisted multilingual processing.
Ollama	Provides the local LLM inference interface.
Domain knowledge base	Supplies external welfare and micro-loan information for retrieval.

Table 7 Software Technologies and Their Roles in the SmartGrama NLP Component

3.3 Experimental or Study Procedure

The experimental procedure is designed to evaluate how effectively the implemented pipeline processes a question, retrieves domain-specific evidence, and generates a response. The procedure is divided into knowledge preparation, semantic indexing, multilingual query processing, contextual generation, and question-based evaluation.

3.3.1 Knowledge Collection and Preparation

The first stage involves collecting and organising the domain information available to the SmartGrama NLP component. The information is reviewed to identify content relevant to welfare and micro-loan enquiries.

The preparation process aims to make the source material suitable for machine retrieval. Where necessary, text is cleaned to remove irrelevant formatting, redundant whitespace, and other content that may interfere with processing. The documents are then organised into text segments that preserve meaningful information.

Segmentation is necessary because retrieval systems typically work with smaller passages rather than entire collections of documents. If a segment is excessively large, it may include irrelevant information and make contextual generation less focused. If it is too small, it may omit important details needed to interpret a requirement or procedure.

The objective is therefore to create segments that contain sufficiently coherent information for retrieval and subsequent answer generation. The precise segment length, overlap, and preprocessing configuration should be documented from the actual implementation rather than assumed.

3.3.2 Semantic Embedding Generation

After preparing the knowledge segments, the system uses Sentence Transformers to represent text as numerical vectors called embeddings. These vectors encode semantic information in a form that supports similarity comparisons.

Each knowledge segment is converted into an embedding during the indexing stage. When a user submits a question, the system generates an embedding for the query using the corresponding embedding model and processing configuration.

Semantic embeddings help the system identify related information even when the question and the stored passage do not use exactly the same words. For example, a citizen might ask about documents needed for a loan, while a knowledge passage describes the documentation required for an application. Semantic similarity may help connect these expressions when their meanings are sufficiently related.

The effectiveness of this stage depends on the embedding model, language characteristics, text preparation, and the compatibility between query and document representations. Translation-assisted processing is particularly relevant when the query language differs from the language or representation used by the stored knowledge.

3.3.3 Vector Indexing and FAISS Retrieval

The knowledge embeddings are indexed using FAISS to support similarity based search. When a query embedding is generated, the retrieval process compares it with the indexed vectors and identifies the passages considered most relevant according to the configured search method.

The retrieved passages provide the external evidence used in the subsequent generation stage. This process is important because the language model should receive information that is directly related to the user's question rather than relying exclusively on general knowledge.

The retrieval procedure can be represented as:

User question → Query embedding → FAISS similarity search → Ranked or selected knowledge passages.

The exact similarity metric, index configuration, number of retrieved passages, and retrieval threshold must be reported according to the implementation. These parameters should not be assigned arbitrary values in the thesis.

Retrieval quality is evaluated by checking whether the returned passages contain information relevant to the question and whether they provide an appropriate basis for answering it.

3.3.4 Multilingual Query Processing

The NLP component is designed to support English and Sinhala interaction. Multilingual processing is necessary because citizens may express the same information need using different languages and wording.

The pipeline processes the incoming query according to the implemented language-handling path. Translation is used where required to support retrieval or response generation. The `deep-translator` library provides the translation functionality used in the component.

Translation-assisted processing can improve practical language coverage, but it may also introduce errors. A translated query may lose a qualification, alter a technical term, or change the meaning of an administrative expression. Similar issues may arise when an answer is translated into the user's preferred language.

For this reason, language support is evaluated by examining whether English and Sinhala questions preserve their intended meaning through processing and whether the final responses communicate the required information clearly.

The system is not described as a newly trained native Sinhala foundation model. Its multilingual functionality is based on the implemented language processing and translation-assisted approach.

3.3.5 Retrieval Augmented Response Generation

After retrieving relevant knowledge passages, the backend constructs a context containing the user's question and the selected information. This context is passed to the language model through the Ollama inference interface.

The general response generation process is represented as:

Generated response = LLM (user question, retrieved context).

The objective is to generate a natural language answer grounded in the available domain evidence. By providing retrieved passages to the model, the system attempts to reduce dependence on unsupported internal knowledge and improve the relevance of answers to SmartGrama services.

However, RAG does not guarantee that every generated statement is correct. The model may omit important conditions, misinterpret retrieved information, introduce unsupported details, or produce an answer that does not fully address the question.

The evaluation therefore examines the generated answer alongside the retrieved context and expected answer. A response is not considered fully reliable simply because a retrieval operation returned relevant passages.

3.3.6 Integration with the Conversational Interface

The React frontend provides the user facing interface through which questions are submitted and responses are displayed. The Flask backend coordinates the query-processing workflow and connects the interface with the embedding, retrieval, translation, and generation components.

The overall interaction follows these steps:

1. The user submits a question in English or Sinhala through the interface.
2. The frontend sends the query to the backend.
3. The backend performs the applicable language-processing steps.
4. The query is represented as an embedding.
5. FAISS retrieves relevant passages from the indexed knowledge base.
6. The backend combines the question and retrieved context.
7. Ollama is used to generate a context informed response.
8. Translation or other language-handling steps are applied when required.
9. The response is returned to the frontend for display.

This integration allows the NLP component to function as a conversational information-access layer within SmartGrama. It does not replace the platform's formal identity-verification, loan risk assessment, or welfare eligibility decision-making functions.

3.3.7 Question Based Evaluation Procedure

The 217 question dataset is used to assess the integrated system. For each question, the available evaluation record contains the question, expected answer, generated answer, retrieved context, language, category, and response time.

The evaluation procedure involves the following activities:

1. Load or prepare the structured evaluation questions.
2. Submit each question to the NLP pipeline using the configured processing path.
3. Record the generated answer and relevant retrieved context.
4. Record response time using the existing evaluation procedure.
5. Compare the generated answer with the expected answer.
6. Inspect the retrieved context to determine whether it supports the response.

7. Assess relevance, correctness, completeness, groundedness, and language quality.
8. Group the findings by language and question category where the available records permit.
9. Review incorrect or incomplete responses to identify recurring failure patterns.
10. Summarise the findings and document limitations.

The dataset contains 217 question records. This number represents the size of the evaluation dataset, not the number of human participants. The final analysis should report the number of successfully processed records and any missing or failed runs if these are present in the actual evaluation logs.

3.4 Measures and Evaluation Metrics

The evaluation uses multiple measures because no single metric can fully describe the performance of a multilingual RAG system. Retrieval relevance, response correctness, groundedness, completeness, language quality, and response time represent different aspects of system behaviour.

3.4.1 Retrieval Relevance

Retrieval relevance measures whether the passages returned by FAISS contain information related to the user's question. It is assessed by examining the retrieved context against the question and the expected information needed to answer it.

A relevant passage should contribute directly to resolving the information request. A passage that shares a few words with the question but concerns a different procedure should not automatically be considered relevant.

Where relevance labels are assigned to individual retrieved passages, a retrieval metric can be calculated using the documented relevance-judgement procedure. For example, Precision at (k) can be used when the top (k) retrieved passages are available and their relevance has been judged.

This metric should only be reported numerically if the necessary passage-level relevance judgements and retrieval records are available. The presence of retrieved context alone does not establish a retrieval-accuracy percentage.

3.4.2 Answer Correctness

Answer correctness measures whether the generated response provides information consistent with the expected answer and the available domain evidence.

The assessment considers whether the answer states the correct facts, avoids contradicting the source material, and does not introduce misleading information. Responses should be assessed with particular care when discussing eligibility conditions, required documents, financial terms, or application procedures.

If answers are classified using predefined labels such as correct and incorrect, the correctness rate can be calculated as:

The number of evaluated answers must be used as the denominator. If some records are missing or excluded, this should be stated explicitly.

3.4.3 Response Relevance

Response relevance assesses whether the generated answer directly addresses the question. A response may contain generally correct information but still be unsuitable if it discusses an unrelated service, provides unnecessary details, or fails to answer the central request.

The assessment considers whether the answer remains focused on the user's information need and whether the information is appropriate to the question category.

3.4.4 Groundedness

Groundedness measures the extent to which claims in the generated answer are supported by the retrieved context. This is a central evaluation dimension for a RAG-based assistant.

An answer may be factually plausible but unsupported by the retrieved evidence. Such an answer should be distinguished from a response that is both correct and supported by the available source passages.

The evaluation therefore checks whether important claims can be traced to the retrieved context. Unsupported URLs, unverified procedures, invented requirements, and additional claims not present in the evidence are potential grounding failures.

Groundedness assessment should be based on inspection of the actual question, retrieved passages, and generated answer. It should not be inferred merely from the fact that FAISS returned a result.

3.4.5 Answer Completeness

Completeness measures whether the response covers the key information required to answer the question. An answer may be correct but incomplete if it omits an essential condition, qualification, step, or limitation.

The expected answer is used as a reference for identifying the important information elements. The assessment then determines whether the generated response includes those elements without introducing contradictory or unsupported details.

Completeness is particularly important for procedural questions, where omitting a required step or condition may make otherwise accurate guidance misleading.

3.4.6 English and Sinhala Language Quality

Language quality assesses whether the generated response is understandable and preserves the intended meaning of the question and relevant source information.

For English responses, the assessment considers clarity, grammatical readability, terminology, and relevance. For Sinhala responses, the assessment considers readability, appropriate Sinhala wording, preservation of technical meaning, and the effect of translation on administrative or financial terminology.

The evaluation also considers whether the response uses the intended output language. Language quality and factual correctness are assessed separately because a fluent answer may still contain incorrect information, while a factually relevant answer may be difficult to understand because of translation problems.

3.4.7 Response Time

Response time measures the time required to process a question and return a response according to the implemented evaluation procedure.

The response time may be influenced by query preprocessing, translation, embedding generation, vector retrieval, LLM inference, local hardware, and the length of the generated answer. Local model loading and warm up behaviour may also affect measurements depending on how timing was implemented.

If the timing records are available, the median and a high percentile such as the 95th percentile can provide additional information about typical and slower responses. The timing method, measurement units, and treatment of failed requests must be documented before comparing results.

Evaluation measure	Main assessment question	Evidence required
Retrieval relevance	Did the system retrieve information relevant to the question?	Query and retrieved passages
Answer correctness	Is the response consistent with the expected answer and source evidence?	Expected and generated answers
Response relevance	Does the answer address the user's actual question?	Question and generated answer
Groundedness	Are the answer's claims supported by retrieved context?	Retrieved context and generated answer
Completeness	Does the response cover the required information?	Expected and generated answers
Language quality	Is the response understandable and appropriate in English or Sinhala?	Question, language label, and generated answer
Response time	How long did processing take under the recorded conditions?	Valid response-time observations

Table 8 Software Technologies and Their Roles in the SmartGrama NLP Component

3.4.8 Evaluation Consistency

To improve consistency, the evaluation criteria should be defined before responses are judged. The same criteria should be applied across both languages and across the different question categories.

Where manual assessment is used, the evaluator should record the reason for each incorrect, incomplete, or unsupported response. If more than one evaluator is involved, the assessment procedure and method for resolving disagreements should be documented.

Any automated evaluation measure should also be described accurately, including its calculation method and limitations. Automated text similarity should not be treated as a complete substitute for factual verification, particularly when two answers express the same meaning using different wording.

3.5 Data Analysis

The collected evaluation records are analysed to understand the behaviour of the NLP component and identify areas requiring improvement. The analysis combines descriptive statistics with qualitative inspection of selected responses and failure cases.

3.5.1 Descriptive Analysis

Descriptive analysis summarises the characteristics of the evaluation dataset and the observed system outputs. Relevant summaries include the number of question records, distribution by language, distribution by category, availability of generated answers, availability of retrieved context, and completeness of response-time records.

Where the underlying records support calculation, correctness and other classified evaluation outcomes can be reported as counts and percentages. Response time can be summarised using suitable descriptive statistics.

The results should distinguish between the full dataset size and the number of valid observations for each metric. For example, if response times are missing for some questions, the mean response time must be calculated only from valid timing records, and the number of observations should be reported.

3.5.2 Language Based Analysis

The evaluation is examined by language to identify differences between English and Sinhala interactions. This analysis considers retrieval relevance, correctness, groundedness, completeness, language quality, and response time where the required records are available.

Language based analysis is important because translation-assisted processing may affect query meaning and terminology. However, differences should not be attributed to language alone without considering other factors, such as question difficulty, knowledge base coverage, and the number of questions in each language group.

If the two language groups contain different numbers or types of questions, their results should be interpreted cautiously rather than compared as if they were perfectly balanced samples.

3.5.3 Category Based Analysis

The dataset is also examined by question category where the recorded labels allow this analysis. The purpose is to identify whether certain types of information requests are more difficult for the assistant.

For example, questions requiring a simple definition may behave differently from questions requiring several procedural steps or specific eligibility conditions. Category-based analysis can help distinguish a general retrieval problem from a knowledge-coverage problem affecting only particular topics.

The final thesis should use the category names and counts recorded in the dataset rather than introducing categories that are not represented in the evaluation records.

3.5.4 Error Analysis

Error analysis is used to investigate responses that are incorrect, incomplete, irrelevant, poorly grounded, or linguistically unclear. Each selected failure case is examined by comparing the original question, retrieved context, expected answer, and generated answer.

The analysis distinguishes several potential failure types:

- **Retrieval mismatch:** The retrieved passages do not contain the information needed to answer the question.
- **Insufficient knowledge coverage:** The required information is absent or incomplete in the knowledge base.
- **Incomplete response:** The retrieved evidence may be relevant, but the generated answer omits important details.
- **Unsupported generation:** The answer introduces claims that are not supported by the retrieved evidence.
- **Translation related error:** Translation changes the meaning of the question, source information, or generated answer.
- **Terminology problem:** The answer uses unclear, inaccurate, or unsuitable administrative or financial terminology.
- **Response or integration failure:** The system fails to return a usable response or encounters an issue during processing.

These categories are used as an analytical framework. They should only be reported as observed failure patterns when supported by the actual evaluation records or implementation evidence.

3.5.5 Interpretation of Findings

The findings are interpreted in relation to the research objectives. The analysis considers whether the component can connect multilingual queries with relevant domain knowledge,

whether retrieved context supports answer generation, and which limitations affect the reliability of responses.

The interpretation does not assume that RAG automatically removes hallucinations or that successful software execution proves high answer accuracy. Instead, the conclusions distinguish between functional operation, retrieval behaviour, response quality, and broader real-world effectiveness.

The study is primarily a technical evaluation. Unless separate user-study evidence exists, conclusions about citizen satisfaction, usability across the wider population, or real-world social impact are not inferred from the 217-question dataset.

3.6 Ethics, Privacy, Security and Responsible AI

The SmartGrama NLP component is intended to improve access to welfare and micro loan information. Because such information may influence users' understanding of public services and financial procedures, the methodology considers privacy, information reliability, transparency, and responsible use of AI generated responses.

3.6.1 Privacy and Data Minimisation

The assistant should process only the information necessary to answer the user's request. Questions about general welfare programmes or micro loan procedures normally do not require citizens to disclose sensitive personal details.

Personal identifiers and unnecessary personal information should therefore be excluded from evaluation records wherever possible. If personal data is required for a separate function of the wider SmartGrama platform, that processing should be handled under the relevant access controls and data-protection requirements rather than being unnecessarily copied into the NLP prompt or evaluation dataset.

The final implementation should document the actual information collected, stored, logged, and transmitted. Privacy protection should not be claimed solely because a local language model is used.

3.6.2 Local Language Model Processing and External Dependencies

Ollama provides a local inference interface for the selected language model. Local inference can reduce the need to send conversation content to an external commercial LLM service during the core generation stage.

However, the overall privacy characteristics also depend on translation configuration, application logs, network communication, and the storage of evaluation records. Since `deep-translator` may rely on an external translation service depending on its configuration, the implementation must be reviewed to determine whether user queries are transmitted to an external provider.

Accordingly, the thesis distinguishes local LLM inference from the complete data flow of the system. The use of a local LLM should not be interpreted as proof that all processing remains local.

3.6.3 Information Reliability and Human Oversight

The assistant provides informational guidance and should not be presented as an official authority on welfare eligibility, loan approval, or financial decisions. Generated responses must not replace decisions made by authorised institutions or officials.

Where the knowledge base does not contain sufficient evidence, the system should avoid presenting uncertain information as a confirmed requirement. For authoritative decisions, complex cases, or conflicting information, users should be directed to the appropriate official service or human support channel.

This principle is particularly important for statements about eligibility, deadlines, required documents, interest or repayment conditions, and application procedures. Incorrect information in these areas may create practical difficulties for citizens.

3.6.4 Responsible Use of Generated Responses

The system should be assessed for factual support rather than conversational fluency alone. Retrieved evidence should be inspected, and unsupported claims should be recorded as potential failures.

The knowledge base should use appropriate sources and should be updated when service information changes. Where source dates or version information are available, they should be retained to support traceability and maintenance.

The component should also avoid implying that it has performed formal verification or made an official decision when it has only generated an informational response.

3.6.5 Security and Access Control

The application should use appropriate controls for the frontend, backend API, knowledge files, configuration settings, and evaluation records. Credentials and sensitive configuration values should not be embedded in publicly accessible code or included in the thesis.

Logs should be limited to what is necessary for debugging, performance analysis, and research evaluation. Any deployment beyond the local prototype would require further assessment of authentication, API exposure, network security, retention, and access-control requirements.

3.6.6 Research Integrity

The evaluation findings must be reported honestly. The number of questions in the dataset must not be represented as the number of study participants, and no accuracy, satisfaction, or performance percentage should be claimed unless it has been calculated from appropriate evidence.

The methodology and findings should distinguish implemented features from planned improvements. Any use of AI assisted writing or other automated tools must be disclosed in accordance with the applicable SLIIT academic integrity and research reporting requirements.

3.7 Reproducibility and Research Artefacts

Reproducibility is important because the behaviour of a RAG system depends on the knowledge base, embedding model, retrieval configuration, translation pathway, language model, prompts, and software environment.

The research maintains a methodological basis for reproducing the evaluation through the use of a structured question dataset and recorded response information. Where available, the research artefacts include the knowledge base documents, preprocessing procedures, embedding configuration, FAISS index settings, backend implementation, language-processing configuration, Ollama model configuration, evaluation records, and analysis results.

Artefact	Purpose
Knowledge base documents	Identify the external information available to the assistant.
Preprocessing procedure	Explain how source text is prepared for retrieval.
Embedding model and configuration	Identify the method used to represent text semantically.
FAISS index configuration	Document the retrieval setup.
Translation configuration	Describe the language processing pathway.
Ollama model configuration	Identify the model and inference settings used in testing.
Evaluation question dataset	Provide the questions used for technical assessment.
Generated answers and retrieved context	Support inspection of the RAG pipeline's behaviour.
Response time records	Support analysis of processing time.
Evaluation criteria and analysis	Explain how outcomes and failure cases are assessed.

Table 9 Research Artefacts and Their Purposes in the SmartGrama NLP Component

To support repeatability, the final thesis should identify exact software versions and configuration values from the actual development environment wherever they are available. The embedding model name, selected Ollama model, FAISS index type, retrieval parameters, text chunking settings, and timing method should be reported from the implementation rather than guessed.

Where possible, the evaluation should be repeated using the same configuration and dataset. If the model, knowledge base, or retrieval settings change, the change should be documented because it may affect the generated responses and measured performance.

Reproducibility is also subject to practical limitations. LLM output may vary with model configuration, software versions, and generation settings, while translation services and knowledge sources may change over time. These factors should be considered when interpreting repeated evaluations.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

The requirements of the SmartGrama NLP component were identified based on the need to provide accessible, natural-language information about welfare programmes and micro-loan services. The component is designed to accept questions in English and Sinhala, retrieve relevant information from a domain-specific knowledge base, and generate context-informed responses through a Retrieval Augmented Generation (RAG) architecture.

The design combines a conversational interface, backend orchestration, semantic embedding generation, vector based retrieval, multilingual processing, and large language model inference. Each part has a specific responsibility, allowing the component to be maintained and improved without redesigning the entire system.

The requirements are divided into functional requirements, which describe what the system should do, and non functional requirements, which describe the expected quality and operational characteristics of the system.

4.1.1 Functional Requirements

Functional requirements describe the primary operations provided by the NLP component.

Table 10 Functional Requirements of the SmartGrama NLP Component

Requirement ID	Functional requirement	Description
FR1	Accept natural-language questions	The system accepts questions submitted through the conversational interface.
FR2	Support English and Sinhala	The system supports English and Sinhala interaction through the implemented language-processing pathway.
FR3	Generate query embeddings	The system converts queries into semantic vector representations using Sentence Transformers.
FR4	Retrieve relevant knowledge	The system uses FAISS to retrieve relevant passages from the indexed knowledge base.

Requirement ID	Functional requirement	Description
FR5	Construct an augmented prompt	The backend combines the user's question with retrieved contextual information.
FR6	Generate a response using an LLM	The system uses the Ollama inference interface to generate a response from the question and retrieved context.
FR7	Return the response to the interface	The backend returns the generated response to the React frontend for display.
FR8	Record evaluation information	The evaluation process records relevant information such as questions, expected answers, generated answers, retrieved context, and response time.

These requirements establish the functional scope of the individual component. They do not imply that the NLP assistant independently performs all operations available in the wider SmartGrama platform.

4.1.2 Non Functional Requirements

Non-functional requirements concern the reliability, maintainability, usability, and operational characteristics of the component.

Requirement	Design response
Answer reliability	Use retrieved domain information as context and evaluate generated responses against expected answers and source evidence.
Relevance	Apply semantic embeddings and vector similarity search to identify potentially relevant knowledge passages.
Multilingual accessibility	Provide English and Sinhala query handling through the implemented language-processing and translation-assisted workflow.
Maintainability	Separate the frontend, backend, retrieval logic, and generation responsibilities into manageable components.
Performance	Use FAISS for vector similarity search and measure end-to-end response time under the recorded test conditions.
Privacy	Use local LLM inference where configured and minimise unnecessary personal information in queries, prompts, and logs.
Knowledge management	Maintain a domain-specific knowledge base that can be reviewed and updated as service information changes.
Responsible AI	Avoid treating generated responses as official decisions and provide a suitable fallback when reliable information is unavailable.

Table 11 Non-Functional Requirements and Design Responses

The use of semantic retrieval and local LLM inference supports the intended design goals, but it does not automatically guarantee high accuracy, complete privacy, or a particular response time. These properties require implementation-level verification and evaluation.

4.2 Component Architecture Within the Complete Solution

The SmartGrama NLP component acts as a conversational information-access layer within the wider SmartGrama platform. Its main responsibility is to help users obtain relevant welfare and micro-loan information through natural-language interaction.

The architecture follows a layered design. The React frontend handles user interaction, the Flask backend coordinates processing, the semantic retrieval layer identifies relevant knowledge, and the generation layer produces a response using the retrieved context. The domain knowledge base supplies the information used by the retrieval process.

4.2.1 High Level Architecture

The main processing flow can be represented as follows:

User → React Interface → Flask Backend → Language Processing → Sentence Transformers → FAISS Retrieval → Retrieved Context → Ollama LLM → Response → React Interface → User

The knowledge base is connected to the retrieval layer through the indexing and search process. Its documents are prepared and represented as vectors before they can be retrieved efficiently during a conversation.

The architecture separates the responsibilities of retrieving information and generating a natural language answer. This separation is important because retrieving a relevant passage and producing a correct response are different tasks. The retrieval stage identifies supporting information, while the generation stage interprets that information in relation to the user's question.

4.2.2 Architectural Layers

The architecture consists of the following logical layers.

Layer	Main responsibility	Technology or resource
Presentation layer	Accept questions and display responses.	React
Application layer	Receive requests and coordinate NLP processing.	Flask and Python
Language-processing layer	Handle English and Sinhala processing and translation where required.	deep-translator and application logic
Embedding layer	Represent queries and knowledge passages as semantic vectors.	Sentence Transformers

Layer	Main responsibility	Technology or resource
Retrieval layer	Search the indexed vectors and return relevant passages.	FAISS
Generation layer	Generate a response using the question and retrieved context.	Ollama and the selected LLM
Knowledge layer	Store the domain information used to answer questions.	SmartGrama knowledge base
Evaluation layer	Organise questions and record outputs for technical assessment.	Evaluation dataset and analysis tools

Table 12 Architecture Layers and Their Responsibilities

These layers represent logical responsibilities rather than necessarily separate physical servers or independently deployed services. The actual deployment arrangement depends on the implemented environment.

4.2.3 Role of the NLP Component in SmartGrama

The NLP component provides a conversational method for accessing information that may otherwise require users to search through documents or interpret administrative terminology.

For example, a citizen may ask about the information required to apply for a micro-loan or request details about an available welfare programme. The component processes the question, searches its knowledge base for relevant information, and generates a response using the retrieved context.

The NLP component remains separate from other system responsibilities, including blockchain based verification, identity management, credit risk analysis, and formal eligibility decisions. This separation helps establish a clear boundary between informational guidance and operations that require separate rules, data, or authorisation.

4.3 Design Alternatives and Decisions

Several approaches could be used to implement a conversational information-access system. The design decisions for SmartGrama were made around the need to handle natural-language variation, provide domain-specific information, support English and Sinhala interaction, and integrate with the available software environment.

The selected architecture combines semantic retrieval with LLM generation rather than relying exclusively on keyword matching or a general purpose LLM.

4.3.1 Rule Based Responses Versus RAG

A rule-based chatbot can provide predictable answers to predefined questions. However, it requires developers to define rules or response patterns for the supported inputs. This can make the system less flexible when users express the same information need in different ways.

A general-purpose LLM can interpret a wider range of questions, but it may generate unsupported or inaccurate details when responding without appropriate domain evidence.

RAG combines retrieval with generation. It retrieves relevant information from an external knowledge base and supplies that information to the language model. This makes it a suitable design for a domain-specific assistant whose information sources may change over time.

4.3.2 Keyword Retrieval Versus Semantic Retrieval

Keyword retrieval identifies information using matching terms. It can work well when a question uses the same vocabulary as the stored documents, but may fail when users express an idea using different wording.

Semantic retrieval represents queries and documents as vectors and compares their representations. This approach can help identify related passages even when the exact words differ.

For SmartGrama, Sentence Transformers and FAISS provide the semantic embedding and similarity search components. Their suitability still depends on the selected embedding model, language handling, and the quality of the knowledge segments.

4.3.3 Local LLM Versus External LLM Service

The design uses Ollama to provide an interface for local LLM inference. Local inference can offer greater control over the model execution environment and reduce the need to send conversational content to an external commercial LLM service during the core generation stage.

However, local inference requires suitable computing resources, and response time depends on the model and the available hardware. It also does not eliminate external dependencies elsewhere in the pipeline, such as a translation service that may be used by the configured translation library.

The choice therefore supports local control of generation, while privacy and performance remain properties that must be assessed in the complete deployment.

4.3.4 Translation Assisted Multilingual Processing

A multilingual assistant must process questions expressed in English and Sinhala. One possible approach is to use a model with suitable multilingual capabilities directly. Another is to use translation assisted processing to convert a query into a representation compatible with the retrieval workflow.

The SmartGrama component uses the translation-assisted approach described in the implementation. This provides a practical route for bilingual interaction without requiring the research to train a new Sinhala foundation model.

The approach introduces possible translation errors and terminology inconsistencies. Consequently, the design includes language-quality assessment as part of the evaluation process.

Table 13 Design Alternatives and Selected Decisions

Design area	Alternative considered	Selected design	Reason
Conversational response	Rule-based answers or RAG	RAG with an LLM	Combines natural-language generation with retrieved domain evidence.
Information retrieval	Keyword matching or semantic search	Sentence Transformers and FAISS	Supports meaning-based similarity retrieval.
Language generation	External generation service or local inference	Ollama-based local inference	Provides local model execution and greater control over core generation.
Multilingual processing	Translation-assisted processing or a fully native multilingual model	Translation-assisted English/Sinhala processing	Matches the implemented bilingual processing approach.
Knowledge management	Embed information directly in prompts or maintain an external knowledge base	Domain-specific knowledge base with indexing	Supports retrieval and maintenance of domain information.
Application structure	Combine all logic in one interface or separate responsibilities	React frontend and Flask backend	Separates presentation from backend processing.

The alternatives in this table explain the design rationale; they should not be interpreted as evidence that formal comparative experiments were performed between every alternative.

4.4 Detailed Design

This section describes the principal processing stages of the RAG pipeline. It explains how domain information is prepared for retrieval, how a user query is processed, how relevant passages are selected, and how the final response is generated.

4.4.1 Knowledge Ingestion and Indexing

Knowledge ingestion is the process of preparing domain information for retrieval. Source documents are organised and processed into text that can be searched by the NLP component.

The logical workflow is:

Source documents → Text extraction and cleaning → Text segmentation → Embedding generation → FAISS indexing

During text segmentation, documents are divided into smaller passages that preserve meaningful units of information. Each passage is then represented by an embedding generated with Sentence Transformers.

The generated vectors are stored in a FAISS index. The implementation must preserve a mapping between each indexed vector and its corresponding source passage so that the retrieval process can return the actual text associated with a selected vector.

The knowledge base and its index should be treated as related but distinct resources. The knowledge base contains the source information, while the vector index provides a searchable representation of that information.

When source documents are updated, the indexing process may need to be repeated so that the retrieval index reflects the current content. A production implementation should also consider source versioning and review procedures to reduce the risk of retrieving outdated guidance.

4.4.2 Semantic Retrieval

Semantic retrieval identifies knowledge passages that are relevant to a submitted question. The query is converted into an embedding using the configured Sentence Transformers model and compared against the vectors stored in FAISS.

The retrieval stage returns the selected passages according to the configured similarity-search method. These passages are then made available to the generation stage.

The process can be represented as:

[
q = E(Q)
]

where (Q) represents the processed user query and (E) represents the configured embedding function.

For a collection of indexed document vectors (d_1, d_2, \ldots, d_m), the retrieval mechanism identifies the candidates considered most similar to (q) under the selected search configuration.

The exact similarity measure and ranking method depend on the actual FAISS implementation. Therefore, the final technical documentation should specify the configured index and similarity method rather than assuming a particular metric.

Semantic retrieval improves the system's ability to locate potentially related passages, but it cannot guarantee that the selected context contains every fact needed to answer the question. Retrieval quality must therefore be evaluated independently of response generation.

4.4.3 Multilingual Processing

The multilingual design supports questions in English and Sinhala through the implemented language-handling pathway. The processing stage determines how the query is prepared for embedding and retrieval, applying translation where required.

The general logical workflow is:

User query → Language handling → Translation when required → Query embedding → Retrieval

After the retrieved context has been used for generation, the response is returned in the intended language through the implemented response-processing pathway.

This design is intended to make the assistant accessible to users who prefer either supported language. However, translation-assisted processing may alter the meaning of specific terms, particularly administrative names, eligibility conditions, and financial terminology.

The design therefore treats multilingual support as a complete query-to-response requirement rather than simply a translation of the user's initial question. Both the meaning of the query and the clarity of the final answer must be considered.

4.4.4 Contextual Response Generation

The response-generation stage combines the user's question with the retrieved knowledge passages. The Flask backend prepares the contextual input and sends it to the selected model through the Ollama interface.

The conceptual operation is:

[
R = L(Q, C)
]

where:

- (R) is the generated response.
- (L) is the selected language model.
- (Q) is the user's question.
- (C) is the retrieved contextual information.

The model uses the question to identify the user's information need and the context to support its response. This design encourages answers to be based on the SmartGrama knowledge base instead of relying only on the model's general learned knowledge.

The quality of the response depends on the quality of the retrieved context and the model's ability to interpret it correctly. The model may still produce unsupported details, omit important qualifications, or misunderstand a passage.

For this reason, the design supports evaluation of the question, retrieved context, and generated answer as a connected set. A generated response should not be considered reliable merely because it is grammatically correct or confidently worded.

4.4.5 Error Handling and Fallback Design

The component must account for situations in which it cannot provide a reliable response. These situations may occur when the knowledge base does not contain the requested information, the retrieved passages are irrelevant, translation fails, the backend encounters an error, or the LLM does not return a usable answer.

The intended fallback behaviour is to avoid presenting unsupported information as a confirmed fact. Where evidence is insufficient, the assistant should communicate the limitation and, where appropriate, direct the user to an official information source or human support channel.

.4.4.6 Backend Request and Response Flow

The Flask backend acts as the orchestration layer between the React frontend and the NLP services. It receives the query, performs the required processing, obtains retrieved context, invokes the language model, and returns the generated answer.

At a logical level, the interaction follows this sequence:

1. The frontend submits the user's question to the backend.
2. The backend validates and prepares the incoming request.
3. The language-processing logic handles English or Sinhala input.
4. The query embedding is generated.
5. FAISS retrieves relevant knowledge passages.
6. The backend constructs the contextual input for generation.
7. Ollama processes the input using the selected local LLM.
8. The result is processed for the intended response language where required.
9. The backend returns the response to the frontend.
10. The evaluation process records the required output information when evaluation logging is enabled.

The exact API endpoint names, request fields, response schema, and error codes should be documented from the actual source code. They should not be inferred from the high-level architecture.

4.5 Standards and Professional Practice

The design follows software engineering principles that support modularity, maintainability, responsible processing, and reliable evaluation. These principles are particularly relevant because the assistant provides information related to welfare programmes and financial services.

4.5.1 Modularity and Separation of Responsibilities

The architecture separates user-interface responsibilities from backend orchestration, retrieval, and language generation. This makes it easier to modify one part of the pipeline without necessarily redesigning every other part.

For example, improvements to the retrieval configuration can be investigated separately from changes to the React interface. Similarly, a change to the selected LLM can be evaluated without changing the fundamental purpose of the knowledge base.

The architecture nevertheless requires compatible interfaces and representations between modules. The embedding model used for queries must be compatible with the vectors in the index, and the backend must be able to interpret the retrieved passages correctly.

4.5.2 API Based Integration

The frontend and backend communicate through an API-based interaction. This separates the presentation layer from the processing logic and allows the backend to coordinate the NLP pipeline independently of the user interface.

The API should validate requests, handle processing failures appropriately, and return a consistent response structure. Detailed endpoint specifications and validation rules should be documented using the actual implementation.

4.5.3 Knowledge Base Governance

The reliability of the assistant depends on the quality and currency of its external information. Knowledge sources should be reviewed, and updates should be incorporated into the retrieval index through a controlled process.

Where possible, the system should preserve source identification and version information. This can help identify the origin of retrieved information and reduce the risk of continuing to use outdated guidance after source documents have changed.

4.5.4 Privacy and Data Minimisation

The system should avoid collecting unnecessary personal information when responding to general information requests. Evaluation records should contain only the information required for the research purpose, and credentials or sensitive configuration values should not be included in publicly accessible code or thesis figures.

Although Ollama supports local inference, the complete data flow must also be considered. In particular, translation configuration, logging, and network communication may introduce additional data handling requirements.

4.5.5 Responsible AI and Factual Grounding

The assistant is designed to provide informational guidance rather than official decisions. Responses about eligibility, documentation, application procedures, and repayment information should be supported by appropriate knowledge sources.

The design therefore treats retrieved evidence as an important part of the generation process and includes evaluation of groundedness, correctness, and completeness. The system should communicate uncertainty when evidence is insufficient rather than inventing requirements or presenting unverified details as official facts.

4.5.6 Verification and Continuous Improvement

The architecture supports systematic testing through the 217 question evaluation dataset. Questions, retrieved context, generated answers, and response-time records can be used to investigate retrieval problems, translation effects, incomplete answers, and unsupported generation.

Improvements should be based on observed evidence. For example, retrieval failures may require changes to knowledge segmentation or indexing, while unsupported responses may require improvements to prompt instructions, evidence handling, or fallback behaviour.

The component should be reevaluated after material changes to the knowledge base, embedding configuration, translation workflow, retrieval settings, or selected LLM because these changes can affect its behaviour.

5.IMPLENTATION INTEGRATION AND TESTING

5.1 Development environment and technology stack

The SmartGrama NLP component was implemented as a multilingual conversational assistant designed to provide information about welfare programmes and micro-loan services. The implementation combines a React-based frontend, a Flask backend, semantic embedding generation, FAISS-based information retrieval, translation-assisted language processing, and Ollama-based local language model inference.

The implementation follows the architecture described in Chapter 4. The frontend handles user interaction, while the backend coordinates query processing, retrieval, and response generation. The knowledge retrieval functionality is separated from the Flask API so that the knowledge indexing and conversational processing can be maintained independently.

The selected technologies support different responsibilities within the complete processing pipeline. Their integration allows a natural-language question to be processed, matched against domain information, and answered using retrieved context.

5.1.1 Software Technologies

Table 14 Technology Stack Used in the SmartGrama NLP Component

Technology	Implementation role
React	Provides the conversational user interface.

Technology	Implementation role
Flask	Implements the backend API and coordinates requests.
Python	Supports backend processing and NLP-related logic.
Sentence Transformers	Generates semantic embeddings for queries and knowledge passages.
FAISS	Performs similarity-based search over indexed vectors.
deep-translator	Supports translation-assisted English and Sinhala processing.
Ollama	Provides the interface for local LLM inference.
Flask-CORS	Supports cross-origin communication between the frontend and backend where configured.
Domain knowledge base	Provides welfare and micro-loan information for retrieval.
Evaluation dataset	Provides the 217 question records used for technical evaluation.

The technologies are integrated to provide a complete conversational workflow. The implementation does not involve training a new foundation language model or developing a new vector-search algorithm. Its main technical contribution is the integration of these technologies into a domain-specific bilingual RAG application.

5.1.2 Development Environment

The development environment consists of the software required to run the React interface, Flask API, embedding model, retrieval components, and Ollama inference service.

The backend requires a compatible Python environment and the libraries used by the NLP pipeline. The frontend requires a compatible JavaScript development environment. The selected embedding model and local LLM must also be available to their respective components.

The prototype was operated locally during development. The available implementation evidence confirms successful activation of the backend and loading of the embedding model. The overall architecture also connects the frontend, semantic retrieval, and Ollama-based generation into the intended processing workflow.

Table 15 Development Environment Requirements

Component	Requirement
Frontend environment	JavaScript runtime and React development dependencies.

Component	Requirement
Backend environment	Python runtime and required backend packages.
Embedding environment	Sentence Transformers library and the configured embedding model.
Retrieval environment	FAISS library and the indexed knowledge vectors.
Translation environment	deep-translator and its configured translation pathway.
LLM environment	Ollama and the selected locally available model.
Knowledge files	Source information and retrieval-related data required by the component.
Evaluation environment	The structured evaluation dataset and tools used to record and analyse results.

The exact operating-system version, Python version, Node.js version, package versions, embedding model identifier, Ollama model identifier, and hardware specifications should be added from the actual development environment. They should not be estimated because they may affect reproducibility and performance.

5.2 Component Implementation

The NLP component is implemented through several connected modules. The frontend provides the interaction interface, the Flask backend coordinates the processing pipeline, the retrieval module identifies relevant domain passages, and the LLM module generates a response using the retrieved context.

5.2.1 Frontend Implementation

The frontend is developed using React. It provides a conversational interface through which users can submit questions about welfare programmes and micro-loan services.

The interface includes a message input area, a send control, response display, language selection, microphone interaction, and suggested questions, as described in the project implementation documentation. These elements support interaction without requiring the user to understand the internal retrieval or language generation process.

When a user submits a question, the frontend sends the request to the Flask backend through an HTTP request. The backend processes the query and returns the generated response, which is then displayed in the conversation interface.

The frontend is primarily responsible for presentation and interaction. Semantic embedding generation, retrieval, and LLM inference are coordinated by the backend and the associated NLP modules.

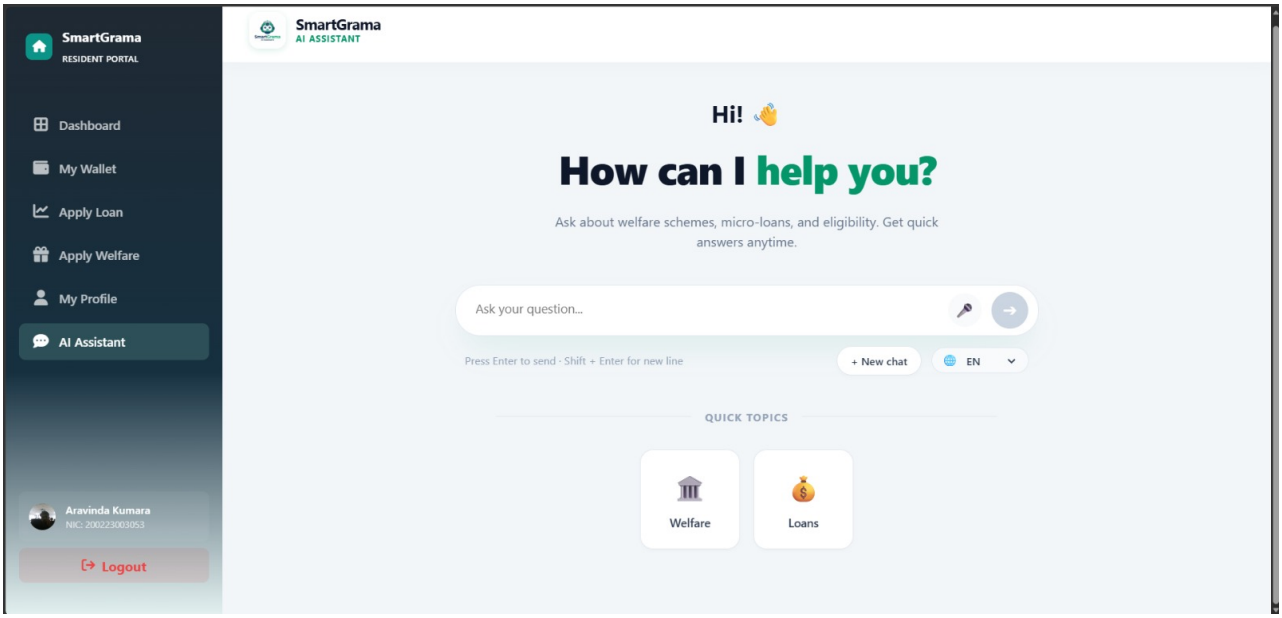


Figure 2 SmartGrama NLP Frontend

5.2.2 Backend Implementation

The backend is implemented using Flask and Python. It acts as the central coordination layer between the frontend and the NLP processing components.

When a query arrives, the backend processes the request through the applicable language-handling pathway, generates the query embedding, retrieves relevant knowledge passages, and supplies the question and retrieved context to the Ollama based language model.

The generated response is then processed as required by the configured multilingual workflow and returned to the frontend.

The request-processing sequence is represented as follows:

1. Receive the user query through an HTTP request.
2. Process the query according to the applicable language pathway.
3. Generate a semantic embedding.
4. Search the FAISS index.
5. Retrieve relevant knowledge passages.
6. Combine the query with the retrieved context.
7. Submit the contextual input to Ollama.
8. Generate the response.
9. Translate the response when required by the implemented workflow.
10. Return the response to the frontend.

Flask CORS is used where configured to support communication between the React frontend and Flask backend when they operate under different origins. The exact endpoint names, request fields, response structure, and error-handling mechanisms should be documented from the actual source code.

5.2.3 Knowledge Retrieval Implementation

The knowledge retrieval functionality is maintained separately from the main Flask API. This separation allows the knowledge-processing and indexing logic to be managed independently of conversational request handling.

The retrieval pipeline uses Sentence Transformers to represent text as vectors and FAISS to search the indexed representations.

During knowledge preparation, the source information is cleaned and divided into text segments. The segments are converted into embeddings and indexed. Each indexed vector must remain associated with the relevant source passage so that the retrieval process can return readable information rather than only numerical vector identifiers.

When the user submits a question, the query is embedded using the compatible embedding model. FAISS searches the index and returns selected passages according to the configured retrieval method. These passages become the contextual information supplied to the language model.

The separation between retrieval and API orchestration supports maintainability. For example, the knowledge base can be updated and re-indexed without necessarily changing the frontend, provided that the interface between the retrieval module and the backend remains consistent.

The retrieval implementation does not guarantee that every returned passage is relevant or sufficient. The retrieved context must therefore be examined during evaluation.

5.2.4 Ollama Based Language Model Implementation

Ollama provides the inference interface used by the component to run the selected language model locally. The Flask backend sends the user's question together with the retrieved domain context to the model and receives the generated response.

The use of retrieved context is important because the assistant operates within a specific information domain. Instead of relying exclusively on the model's internal learned knowledge, the generation stage receives relevant information retrieved from the SmartGrama knowledge base.

The response-generation operation can be represented as:

[
R = L(Q,C)
]

where (R) represents the generated response, (L) represents the selected language model, (Q) is the user query, and (C) is the retrieved context.

Local inference provides greater control over the core generation environment and can reduce the need to transmit conversational content to an external commercial LLM API. However, this does not prove that every part of the application operates locally. The configured translation service, network communication, and application logging must also be considered when assessing data handling.

The selected Ollama model name, generation parameters, and inference configuration should be recorded from the actual environment to support reproducibility.

5.2.5 Multilingual Processing Implementation

The component supports English and Sinhala queries through its implemented language-processing workflow. The `deep-translator` library provides translation functionality where required by the retrieval and response-generation process.

The purpose of multilingual processing is to allow users to request information in their preferred supported language while maintaining access to the domain knowledge base.

Translation-assisted processing can introduce meaning changes, particularly for administrative terms, programme names, eligibility requirements, and financial terminology. Therefore, the generated response must be checked for both factual relevance and language quality.

The implementation should be described as translation-assisted bilingual processing rather than as a newly trained Sinhala language model. Its performance depends on the selected translation pathway, the embedding model, the source content, and the generation model.

5.2.6 Integration of the Processing Modules

The frontend, backend, retrieval module, language-processing functionality, and LLM interface work together to provide the conversational workflow.

The React interface accepts the question and displays the result. Flask coordinates the processing sequence. Sentence Transformers generates semantic representations, FAISS identifies potentially relevant passages, and Ollama generates a response from the query and retrieved context.

This modular arrangement supports maintenance and evaluation because each stage has a distinct purpose. It also makes it possible to inspect the retrieved context separately from the generated response when investigating incorrect or incomplete answers.

5.3 Integration with the Other Components

The NLP assistant is intended to operate as the conversational information-access component within the wider SmartGrama platform. Its role is to help citizens obtain information about welfare programmes and micro-loan services using natural-language questions.

The integration provides a user-facing information layer without replacing the responsibilities of other SmartGrama components.

5.3.1 Frontend to Backend Integration

The React frontend communicates with the Flask backend through HTTP requests. This integration enables user input to be transferred to the processing pipeline and generated responses to be returned for display.

The integration depends on a compatible API interface and the correct configuration of the frontend's backend connection. If the backend is unavailable or the request fails, the interface cannot receive a normal generated response.

Frontend-to-backend communication is therefore an important verification area. Testing should check whether a submitted query reaches the backend, whether the backend returns the expected response structure, and whether the frontend displays the returned answer appropriately.

5.3.2 Backend to Retrieval Integration

The Flask backend communicates with the retrieval module to obtain domain passages relevant to the user's query. The retrieval module returns selected knowledge content that can be incorporated into the generation context.

This connection allows information retrieval to remain separate from request orchestration. It also enables the evaluation process to inspect whether the retrieved passages contain relevant information.

Successful retrieval does not automatically guarantee a correct answer. The retrieved context may be incomplete or unrelated, and the language model may misinterpret otherwise relevant passages.

5.3.3 Backend to LLM Integration

The backend sends the user query and retrieved context to the Ollama inference interface. The model generates a response that the backend subsequently prepares for delivery to the frontend.

The integration requires the selected model to be available and the backend to communicate with the local inference service successfully. Model availability, resource requirements, inference errors, and response time are therefore relevant operational considerations.

5.3.4 Integration Boundary Within SmartGrama

The NLP assistant is limited to conversational information access. It does not independently verify citizen identity, approve micro-loans, calculate authoritative credit-risk decisions, or determine official welfare eligibility.

These boundaries are important because the assistant's output is generated from retrieved information and language-model inference. Any formal decision must remain with the appropriate authorised component or institution.

Table 16 Integration Points and Responsibilities

Integration point	Purpose	Verification focus
React frontend ↔ Flask backend	Transfers queries and responses.	Request delivery and response display.
Flask backend ↔ retrieval module	Obtains relevant knowledge passages.	Retrieval execution and returned context.
Retrieval module ↔ knowledge base	Connects indexed vectors with source text.	Index availability and passage mapping.
Flask backend ↔ Ollama	Sends contextual prompts and receives generated responses.	Model availability and response handling.
Language-processing workflow ↔ translation service	Supports English/Sinhala processing where required.	Translation execution and meaning preservation.
NLP component ↔ wider SmartGrama platform	Provides conversational information access.	Correct separation of informational guidance and formal decisions.

5.4 Verification and Software Testing

Testing is necessary to determine whether the implemented component performs its intended functions and to identify limitations in the retrieval and response-generation process.

The testing approach includes frontend to backend communication testing, semantic retrieval testing, evaluation using the 217 question dataset, inspection of retrieved context, and language-specific assessment of English and Sinhala responses.

The available project documentation confirms that the evaluation dataset contains 217 question records. It also identifies response correctness, relevance, groundedness, completeness, language quality, retrieval behaviour, and response time as evaluation areas. However, exact test pass rates and metric values should only be reported where supported by the recorded results.

5.4.1 Functional Testing

Functional testing checks whether the principal operations of the NLP component work as intended.

Table 17 Functional Test Scenarios

Test ID	Test scenario	Expected behaviour
FT01	Submit an English question.	The question is accepted and processed through the configured pipeline.
FT02	Submit a Sinhala question.	The question is processed through the supported Sinhala pathway.
FT03	Generate a query embedding.	The embedding component returns a representation suitable for retrieval.
FT04	Retrieve domain information.	FAISS returns selected passages from the indexed knowledge base.
FT05	Generate a contextual answer.	Ollama receives the question and retrieved context and returns a generated response.
FT06	Display the generated response.	The frontend receives and displays the backend response.
FT07	Process a question requiring translation.	The configured translation pathway is applied where required.
FT08	Record an evaluation result.	The evaluation process records the relevant output fields when logging is enabled.

The expected behaviours in this table define test scenarios; they do not mean that every scenario has independently passed. The final thesis should include actual outcomes, evidence, and failure details from the testing records.

5.4.2 Frontend to Backend Communication Testing

Communication testing examines whether the React frontend can submit a question to the Flask backend and receive the corresponding response.

The main verification points include request transmission, correct handling of the submitted text, backend response availability, and display of the returned answer.

Where the frontend and backend operate on different origins, the relevant cross-origin configuration must also be checked. Communication errors should be distinguished from retrieval or generation errors because they arise at a different stage of the application.

5.4.3 Semantic Retrieval Testing

Semantic retrieval testing examines whether the indexed knowledge base returns useful passages for domain-specific questions.

The test set should include direct questions and paraphrased questions where available. Paraphrased questions are particularly useful because they test whether the system can identify relevant information when the user's wording differs from the wording used in the source documents.

The retrieved passages should be compared with the question and expected information. Cases where the system retrieves unrelated passages, omits required evidence, or fails to locate information present in the knowledge base should be documented.

The final report should identify the actual retrieval configuration and the method used to judge relevance if numerical retrieval metrics are presented.

5.4.4 RAG Response Testing

RAG response testing examines the relationship between the user question, retrieved context, expected answer, and generated response.

The purpose is to determine whether the language model uses the retrieved information appropriately. The assessment should consider whether the response is factually consistent with the expected answer, directly relevant to the question, sufficiently complete, and supported by the retrieved context.

A relevant retrieved passage does not guarantee a correct generated answer. Therefore, response testing must inspect both retrieval and generation rather than judging only the final text.

5.4.5 English and Sinhala Testing

Language-specific testing examines the quality of the assistant's responses in both supported languages.

English responses are checked for clarity, factual consistency, and relevance. Sinhala responses are examined for readability, preservation of meaning, suitable terminology, and possible translation errors.

Where the same information need is represented in both languages, comparing the outputs can help identify differences in retrieval or answer quality. However, comparisons should account for differences in question wording and dataset composition.

5.4.6 Evaluation Using the 217 Question Dataset

The structured dataset provides a consistent basis for examining system behaviour across multiple questions. Each available record contains fields such as the language, category, question, expected answer, generated answer, retrieved context, and response time.

The evaluation process compares generated answers with expected answers and inspects the retrieved context to identify potential errors. Results can be analysed by language and question category when the relevant labels and valid observations are available.

Table 18 Evaluation Dimensions for the 217 Question Dataset

Evaluation dimension	Verification approach
Correctness	Compare generated answers with expected answers and domain evidence.
Relevance	Check whether the response addresses the user's question.
Groundedness	Determine whether important claims are supported by retrieved context.
Completeness	Check whether required information is included.
Retrieval behaviour	Inspect the relevance and adequacy of retrieved passages.
Language quality	Review English and Sinhala clarity and meaning preservation.
Response time	Analyse valid timing observations using the recorded measurement method.

The dataset size of 217 records is a confirmed characteristic of the evaluation materials. It should not be interpreted as a result indicating that 217 questions were all answered correctly or that every record completed successfully. Those conclusions require examination of the actual evaluation outputs.

5.4.7 Failure Analysis

Failure analysis investigates cases where the system does not provide a satisfactory response. The available project documentation identifies examples of potential failure patterns, including unsupported assumptions, incorrect procedural details, and generated content that extends beyond retrieved evidence.

These issues can arise at different stages of the pipeline. A retrieval mismatch may provide irrelevant evidence, an incomplete knowledge base may lack the necessary information, translation may change the question's meaning, or the LLM may introduce unsupported claims despite receiving relevant context.

5.5 Deployment and Operation

The SmartGrama NLP prototype operates through a React frontend, a Flask backend, local embedding and retrieval components, and Ollama based inference. During local development, these services must be configured so that the frontend can communicate with the backend and the backend can access the required NLP resources.

5.5.1 Local Deployment Arrangement

The frontend provides the user interface, while the Flask service coordinates the request-processing pipeline. The embedding model and FAISS index support retrieval, and Ollama provides the local model inference interface.

The backend must be able to load the required configuration and access the knowledge index before it can process questions successfully. The selected Ollama model must also be available to the inference service.

The implementation documentation reports successful backend activation and embedding-model loading. These are useful indicators of operational readiness, although they do not alone prove that every end to end request succeeds or that every response is correct.

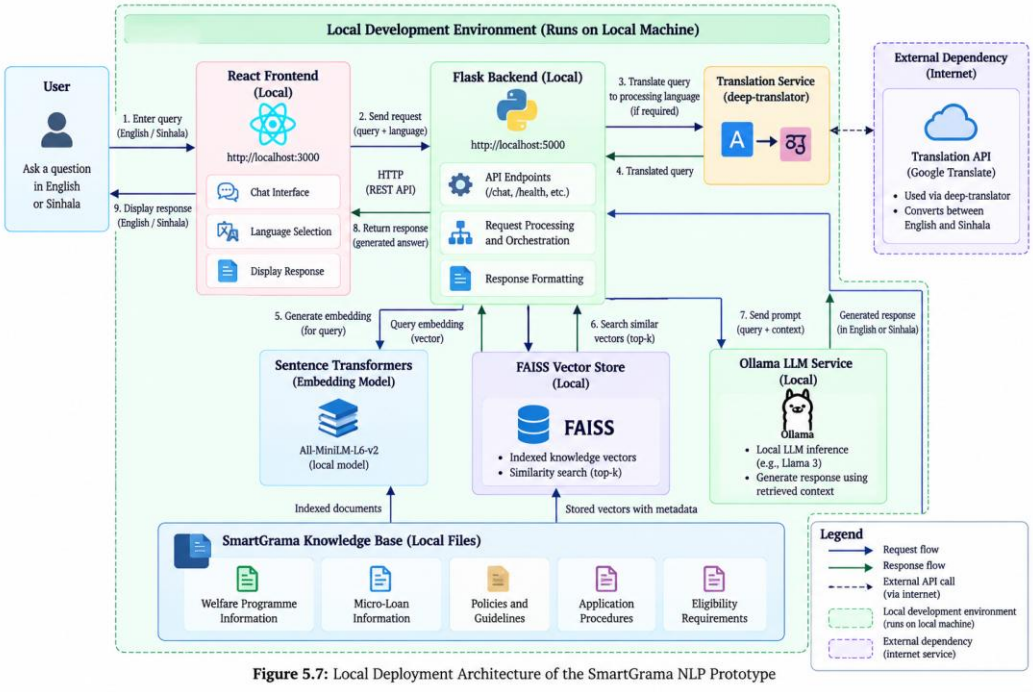


Figure 5.7: Local Deployment Architecture of the SmartGrama NLP Prototype

Figure 3 SmartGrama NLP Local Deployment Arrangement

5.5.2 Operational Workflow

The operational workflow begins when a user submits a question through the conversational interface. The backend processes the query, retrieves domain information, generates a response, and returns the result for display.

For the system to operate correctly, the relevant services and resources must be available. An unavailable backend, missing index, unavailable embedding model, translation failure, or inaccessible LLM can interrupt the request.

The operational process should therefore include checks for service availability and the successful loading of required resources. Where an error occurs, the system should communicate the failure appropriately instead of returning an unsupported answer.

5.5.3 Knowledge Base Maintenance

The knowledge base must be maintained as the underlying welfare and micro-loan information changes. Updating source documents alone may not be sufficient if the indexed vector representations remain based on earlier content.

A maintenance process should review the updated information, preprocess affected documents, regenerate embeddings where required, rebuild or update the FAISS index, and verify that the updated information can be retrieved.

Knowledge updates should preserve the relationship between source passages and their indexed representations. Where possible, source versions and update dates should be recorded to support traceability.

5.5.4 Operational Limitations

The prototype's behaviour depends on the selected embedding model, the quality of the knowledge base, the retrieval configuration, the translation pathway, the selected LLM, and available hardware resources.

Local inference reduces dependence on an external commercial LLM service for core generation but may require substantial local resources depending on the selected model. Translation may also depend on an external service according to its configuration.

The component should consequently be regarded as a research prototype requiring further validation before use as an authoritative public-service information system.

6: RESULTS AND VALIDATION

6.1 Research Results

This chapter presents the results and validation of the multilingual Natural Language Processing (NLP) component developed for the SmartGrama platform. The component combines Retrieval-Augmented Generation (RAG), semantic information retrieval, translation-assisted multilingual processing, and Large Language Model (LLM) response generation to provide conversational access to welfare and micro-loan information.

The evaluation uses a structured dataset containing 217 question records. The dataset includes English and Sinhala questions associated with welfare, micro-loan, eligibility, profile, dashboard, and related information. The available records include expected answers, generated answers, retrieved context, and response-time information.

The evaluation investigates whether the component can retrieve domain-relevant information and use it to generate responses that are relevant, correct, complete, and supported by the available evidence. It also considers multilingual response quality and the operational behaviour of the integrated application.

The results are divided into research results, product and technical results, user or stakeholder validation, and integration results. This structure distinguishes the evidence obtained from question-based evaluation from conclusions about real-world usability or social impact.

6.1.1 Evaluation Dataset

The evaluation dataset provides a structured basis for examining the behaviour of the RAG pipeline. Each record connects the user's question with the expected answer, the generated response, the retrieved context, and the recorded response time.

Table 19 Characteristics of the SmartGrama NLP Evaluation Dataset

Characteristic	Description
Total dataset size	217 question records
Supported languages	English and Sinhala
Main information areas	Welfare, micro-loan, eligibility, profile, dashboard, and related information
Reference information	Expected answer recorded for each evaluation question
Generated output	Response produced by the NLP component
Retrieval evidence	Context returned by the retrieval process
Performance information	Response-time field included in the evaluation records
Evaluation purpose	Assess the behaviour of the multilingual RAG pipeline

The dataset makes it possible to compare generated answers with expected information and inspect the context used during response generation. The availability of retrieved context is particularly important because the system's purpose is to ground its answers in domain-specific information rather than relying exclusively on the language model's general knowledge.

The 217 records represent evaluation questions, not 217 human participants. Therefore, the dataset supports technical question level assessment but does not, by itself, demonstrate user satisfaction or representativeness of the entire population of SmartGrama users.

6.1.2 Evaluation Dimensions

The component is assessed using several complementary evaluation dimensions. Each dimension addresses a different aspect of the pipeline.

Table 20 Evaluation Dimensions and Evidence

Evaluation dimension	Purpose	Evidence examined
Retrieval relevance	Determine whether useful information is retrieved.	User question and retrieved context
Answer correctness	Assess factual consistency with expected information.	Expected and generated answers
Response relevance	Determine whether the response addresses the question.	Question and generated response
Groundedness	Examine whether the response is supported by retrieved evidence.	Retrieved context and generated response
Completeness	Check whether important information is included.	Expected and generated answers
Language quality	Assess English and Sinhala clarity and meaning preservation.	Language label, question, and response
Response time	Examine the time required to return a response.	Recorded timing information

These dimensions are necessary because the quality of a RAG system cannot be described adequately by a single measure. A system may retrieve relevant passages but generate an incomplete answer, or it may produce fluent language that includes unsupported claims.

The available documentation establishes the evaluation framework and the existence of the 217-question dataset. Numerical results for each dimension must be calculated from the corresponding evaluation records before they can be reported.

6.1.3 Retrieval Behaviour

Semantic retrieval is a central component of the SmartGrama NLP architecture. The system uses Sentence Transformers to create vector representations and FAISS to identify potentially relevant passages from the knowledge base.

The retrieved passages provide contextual information to the language model during response generation. The evaluation records include retrieved context, allowing the relationship between the question, retrieved evidence, and generated answer to be examined.

The documented implementation demonstrates that retrieved domain context is available to the generation stage. However, the existence of retrieved context does not establish that every passage is relevant or sufficient. The retrieved passages must be checked against the information required by each question.

The main retrieval-related issues to investigate include:

- Whether the retrieved passages address the actual information need.

- Whether important eligibility conditions or procedural details are present in the retrieved context.
- Whether paraphrased questions retrieve relevant passages despite differences in wording.
- Whether the knowledge base contains the required information.
- Whether translation-assisted processing affects the retrieval result.

A numerical retrieval precision or recall value should only be reported if the necessary relevance judgements and retrieval records are available.

6.1.4 Response Generation Results

The generation stage combines the user query with retrieved context and passes the contextual input to the Ollama based language model. The model produces a response that is returned to the conversational interface.

The available project documentation identifies several response-quality limitations, including unsupported assumptions, incorrect procedural details, and generated claims that extend beyond the retrieved evidence. These findings indicate that retrieval augmentation does not completely eliminate the risk of inaccurate or unsupported generation.

This is an important result for a welfare and micro-loan assistant. An answer may sound convincing while introducing an unverified requirement, providing an incorrect procedure, or omitting a relevant qualification.

Consequently, the generated response must be evaluated against both the expected answer and the retrieved context. The system should not be considered reliable solely because it returns a response or uses a RAG architecture.

6.1.5 Multilingual Results

English and Sinhala interaction is an important objective of the NLP component. The implemented translation-assisted workflow is intended to make welfare and micro-loan information accessible to users who prefer either language.

Multilingual quality must be considered across the complete query-to-response process. Translation may affect the meaning of the original question, the terminology used during retrieval, or the clarity of the final response.

The available documentation establishes that English and Sinhala processing forms part of the implementation and that language-specific inspection is included in the evaluation approach. However, separate numerical performance results for English and Sinhala are not provided in the available summary.

A reliable comparison therefore requires the dataset to be grouped by language and the relevant measures to be calculated independently. The number of questions, valid responses, correctness outcomes, and language-quality assessments should be reported for each language where those records exist.

6.1.6 Response Time Results

Response time is recorded as part of the evaluation dataset. It is relevant because the user experience depends not only on the quality of an answer but also on how quickly the assistant produces it.

The end to end response time may be influenced by query processing, translation, embedding generation, FAISS retrieval, LLM inference, hardware resources, and the length of the generated response.

The available documentation confirms the presence of response-time information in the evaluation records, but it does not provide verified aggregate timing statistics. Therefore, a mean response time, median response time, or 95th-percentile response time should not be stated without calculating it from the actual timing values.

The number of valid observations should be reported alongside the calculated statistic. Failed requests and missing timing records should be handled consistently and documented.

6.2 Product and Technical Results

The implementation provides an integrated architecture connecting the main technologies required for the NLP component. The React frontend provides the conversational interface, Flask coordinates backend processing, Sentence Transformers generates embeddings, FAISS retrieves domain passages, and Ollama provides local LLM inference.

The project documentation confirms successful backend activation and embedding-model loading. It also describes the end-to-end processing workflow through which retrieved context is made available to the generation stage.

Table 21 Product and Technical Results

Component	Result supported by the available documentation
Conversational interface	React-based user interface for submitting questions and displaying responses.
Backend service	Flask-based request processing and orchestration.
Semantic representation	Sentence Transformers used for embedding generation.
Vector retrieval	FAISS used to search indexed knowledge representations.
Multilingual processing	English and Sinhala interaction supported through the implemented language-handling pathway.
Response generation	Ollama used as the local LLM inference interface.
Knowledge grounding	Retrieved domain context supplied to the generation stage.
Evaluation capability	Structured dataset containing 217 question records and associated evaluation fields.

These results demonstrate the technical feasibility of integrating the selected components into a domain-specific conversational application. They do not independently establish a particular accuracy percentage, guarantee that every request succeeds, or prove that the assistant is ready to make authoritative public-service decisions.

The implementation's principal technical contribution is the integration of semantic retrieval, multilingual processing, and context-informed generation for the SmartGrama welfare and micro-loan domain.

6.3 User or Stakeholder Validation

The available validation evidence is primarily based on technical question-level evaluation rather than a statistically designed study involving citizens or service officers.

The 217 question dataset supports systematic inspection of generated responses, retrieved context, and recorded performance information. This makes it possible to identify technical strengths and failure patterns in the implemented pipeline.

However, question-based evaluation is not equivalent to a formal usability study. It does not establish how citizens perceive the interface, whether users can independently complete information-seeking tasks, or whether the assistant improves access to services in real-world settings.

Table 22 Validation Evidence and Its Scope

Validation area	Available evidence	Interpretation
Technical processing	Implemented frontend and backend workflow.	Supports assessment of the software architecture.
Retrieval and generation	Evaluation records include retrieved context and generated answers.	Enables question-level inspection of RAG behaviour.
Multilingual interaction	English and Sinhala are included in the evaluation scope.	Supports language-specific technical assessment when records are grouped appropriately.
Performance	Response-time information is included in the dataset.	Enables timing analysis after calculation from valid records.
User satisfaction	No statistically designed user-study results established by the available evidence.	Satisfaction percentages should not be claimed.
Real-world service impact	No measured population-level impact established by the available evidence.	Social or institutional benefits should be described as potential benefits rather than measured outcomes.

Future validation should include structured feedback from representative citizens and relevant service officers. Such a study could examine ease of use, clarity, trust, language preference, task completion, and the ability to recognise when an answer requires confirmation from an official source.

Any human-participant study should follow the applicable institutional ethics and research procedures.

6.4 Integration Results

The NLP component integrates presentation, backend orchestration, retrieval, multilingual processing, and generation into a single conversational workflow. This enables the user to submit a question through the interface without directly interacting with the underlying NLP technologies.

The principal integration result is the separation and coordination of the component's major responsibilities. React handles interaction, Flask manages the request flow, the retrieval pipeline identifies relevant domain information, and Ollama generates a response using the retrieved context.

Table 23 Integration Assessment of the NLP Component

Integration area	Intended integrated behaviour	Evidence or further validation required
Frontend and backend	Submit questions and return responses.	Actual request and response tests.
Backend and retrieval	Obtain relevant passages for the question.	Retrieved context and retrieval test records.
Retrieval and generation	Supply retrieved evidence to the LLM.	Prompt or execution records where available.
Language processing and response generation	Support the intended English or Sinhala interaction.	Language-specific output inspection.
Evaluation workflow	Associate questions with outputs and timing information.	Completeness and consistency checks on evaluation records.
Wider SmartGrama platform	Provide conversational information access.	Integration evidence from the actual platform environment.

The component is positioned as an information-access layer within SmartGrama. It does not replace identity verification, blockchain based operations, credit risk analysis, or formal welfare and loan decisions.

The available evidence supports the intended integrated architecture. Any claim about complete integration with all other SmartGrama components should be supported by corresponding end-to-end integration tests.

6.5 Summary of Findings

The evaluation and implementation documentation support several conclusions about the SmartGrama NLP component.

First, the project provides a practical multilingual RAG architecture that combines a domain-specific knowledge base, semantic embeddings, FAISS retrieval, translation-assisted English and Sinhala processing, and local LLM inference through Ollama.

Second, the structured 217 question dataset provides a basis for examining generated responses against expected answers and retrieved evidence. It also includes response-time information that can be analysed when the actual measurements are processed.

Third, the implementation demonstrates the intended technical integration of the frontend, backend, retrieval, and generation components. The documented successful activation of the backend and loading of the embedding model provide evidence of operational progress.

Fourth, the available evaluation documentation identifies important limitations, including unsupported assumptions, incorrect procedural details, and generated claims that extend beyond retrieved evidence. These findings show that RAG reduces reliance on unsupported internal knowledge but does not guarantee that all generated responses are correct.

Finally, the available evidence does not establish numerical accuracy, language-specific success rates, aggregate response-time statistics, or user-satisfaction percentages. Such values must be calculated from the evaluation records or collected through an appropriate validation study before they can be reported as research results.

Overall, the findings support the feasibility of a domain-specific English–Sinhala conversational assistant for welfare and micro-loan information access. They also highlight the importance of reliable knowledge sources, retrieval validation, multilingual quality checks, and careful control of generated responses.

7: DISCUSSION

7.1 Interpretation of Findings

The objective of the SmartGrama Natural Language Processing (NLP) component is to improve access to welfare and micro-loan information through a multilingual conversational assistant that supports English and Sinhala. The component combines semantic retrieval, a knowledge base, language-processing functionality, and Large Language Model (LLM)-based response generation within a Retrieval-Augmented Generation (RAG) architecture.

The findings discussed in this chapter are based on the implemented prototype, the documented technical workflow, and the available evaluation dataset containing 217 question records. These records include questions, expected answers, generated answers, retrieved context, language information, categories, and response times. They provide a basis for examining the component's behaviour. However, the existence of these records alone does not establish a particular accuracy rate, statistically significant improvement, or level of user satisfaction.

7.1.1 Effectiveness of the Retrieval Augmented Generation Approach

One of the main design decisions was to use Retrieval-Augmented Generation rather than relying exclusively on an LLM's internally learned knowledge. In the implemented approach, a user submits a question, the system processes the query, and semantic retrieval is used to identify potentially relevant information from the available knowledge base. The retrieved information is then used as context for response generation.

This approach is suitable for the SmartGrama application because welfare and micro-loan questions may concern specific procedures, requirements, eligibility conditions, supporting documents, or repayment-related information. A general-purpose language model may generate fluent answers without necessarily providing information that is specific to the application's knowledge base. Retrieval provides a mechanism for connecting generated answers to relevant domain information.

The implementation uses Sentence Transformers to represent textual information as embeddings and FAISS to support similarity-based retrieval. This allows the system to search for semantically related information rather than depending entirely on exact keyword matches.

Nevertheless, retrieval does not automatically guarantee correctness. If the knowledge base does not contain the required information, if relevant content is not retrieved, or if the retrieved content is incomplete, the generated answer may still be incorrect or incomplete. Therefore, the effectiveness of the RAG approach depends on the quality of the knowledge base, the relevance of retrieved passages, and the ability of the language model to use the supplied context appropriately.

The available evaluation records and implementation evidence support examining this retrieval-and-generation workflow. A quantitative claim that RAG outperforms another approach would require a controlled comparison with an alternative system, which is not established by the currently documented evidence.

7.1.2 Semantic Retrieval and Context Relevance

Semantic retrieval is an important part of the component because users may express the same information need using different words or sentence structures. A system based only on exact keyword matching can struggle when the wording of a question differs from the wording used in the knowledge base.

The use of Sentence Transformers and FAISS provides a technical foundation for finding content based on the similarity of textual representations. This is particularly relevant to conversational questions about welfare services and micro-loan information, where users may ask about application requirements, required documents, or eligibility using different expressions.

However, the relevance of retrieved content must be assessed separately from the fluency of the final response. A response may sound appropriate while being based on context that does not fully answer the user's question. Similarly, relevant information may be retrieved but not incorporated correctly into the generated answer.

The evaluation dataset includes retrieved-context information, which makes it possible to investigate these cases. A useful analysis would classify retrieval outcomes as relevant and sufficient, partially relevant, irrelevant, or insufficient. The classification should be performed against the expected information needed to answer each question rather than solely against textual similarity.

The main implication is that semantic retrieval should be evaluated as an independent stage of the NLP pipeline. This would help distinguish errors caused by retrieval from errors introduced during response generation.

7.1.3 Quality of Generated Responses

The response-generation stage uses an LLM to produce natural-language answers using the information provided through the retrieval process. This makes the interface more conversational than a system that returns only document titles or matching text fragments.

For the SmartGrama use case, response quality involves more than grammatical correctness. A useful answer should address the user's question, preserve the meaning of the retrieved information, avoid unsupported statements, and explain relevant requirements clearly. This is especially important when answers concern public services or financial information, where misleading details may cause confusion.

The documented evaluation material identifies failure patterns such as unsupported URLs or procedures, incorrect terminology, and additional claims that are not supported by the retrieved evidence. These issues demonstrate why answer evaluation should consider factual support and completeness in addition to readability.

For example, an answer may correctly describe a general application process but introduce an unverified website address or an unsupported procedural step. Although the response may

appear helpful, the additional information can reduce its reliability. Similarly, an answer may omit an important requirement even when the retrieved context contains that requirement.

These observations indicate that response generation requires explicit attention to grounding. The system should be assessed according to whether each substantive claim is supported by the available context. Where information is missing, the assistant should ideally communicate that limitation rather than inventing a definitive answer. Such fallback behaviour should be treated as a design requirement unless its implementation has been directly verified.

7.1.4 Multilingual Processing and English Sinhala Support

The component is designed to support English and Sinhala so that users can access welfare and micro-loan information through either language. This is an important aspect of the system's intended accessibility, particularly when users are more comfortable asking questions in Sinhala than in English.

The implementation incorporates language-processing and translation functionality using the `deep-translator` library, together with semantic retrieval and LLM-based response generation. These functions form part of a pipeline intended to process queries and produce understandable responses across the two supported languages.

However, multilingual support should not be interpreted as proof that the system achieves equivalent quality in both languages. Translation may change the meaning of technical terms, eligibility conditions, document names, or financial expressions. A translated question may retrieve different context from an equivalent question in the other language, and the final answer may introduce further terminology or grammatical problems.

Consequently, English and Sinhala responses should be examined separately as well as collectively. A suitable evaluation should compare questions with equivalent information needs across both languages and check whether their answers communicate the same substantive facts. The evaluation should also consider whether important domain terminology is translated consistently.

The available dataset provides a starting point for this investigation because it contains language information and generated answers. Nevertheless, numerical claims about Sinhala accuracy, translation quality, or parity between languages should only be made after calculating the relevant results from the records.

7.1.5 Response Time and Practical Usability

Response time is another important consideration because the system is intended to support interactive conversations. A technically correct answer may still provide a poor user experience if the system takes an unreasonable amount of time to respond.

The evaluation records include response-time information, allowing the time taken to generate answers to be investigated. Actual response-time performance may depend on several factors, including the size of the knowledge base, embedding and retrieval operations, translation processing, the selected language model, available computing resources, and the length of the generated answer.

The use of local Ollama-based inference supports running the language model within the local development environment. However, local execution does not by itself establish that the component will be sufficiently fast or scalable in a production environment.

To interpret response-time results meaningfully, the mean, median, and a suitable upper-percentile response time should be calculated from valid measurements. Results should also be reported separately where English and Sinhala processing paths differ. Any reported measurements should identify the hardware and model configuration used during testing.

Therefore, response time should be considered alongside answer quality rather than as an isolated indicator of system effectiveness.

7.2 Comparison with Literature, Baselines, and Existing Solutions

The proposed NLP component can be discussed in relation to established approaches to conversational systems, information retrieval, language modelling, and RAG. This comparison is conceptual and literature-based. The available evidence does not establish that controlled experiments were conducted against every alternative described below.

7.2.1 Rule Based Conversational Systems

Rule-based conversational systems generally rely on predefined patterns, keywords, decision trees, or fixed responses. They can be appropriate when questions and expected user actions are limited and clearly defined. Their behaviour can also be relatively predictable because responses are determined by explicitly specified rules.

However, rule-based approaches may struggle when users express questions in unexpected ways or combine multiple information needs in one message. Maintaining large numbers of rules can also become difficult as the knowledge domain grows.

The SmartGrama NLP component adopts a different approach by combining semantic retrieval with language-model-based generation. This offers a mechanism for responding to varied natural-language questions while using retrieved domain information to guide the answer. The trade-off is that generated responses require additional validation because fluent language generation can introduce unsupported details.

7.2.2 Keyword Based Information Retrieval

Traditional keyword-based retrieval identifies documents through matching terms or other lexical criteria. This can be effective when users employ the same terminology as the source material. However, differences in wording may reduce the relevance of returned information.

The SmartGrama implementation uses embeddings generated through Sentence Transformers and similarity search supported by FAISS. The intended advantage is that the system can retrieve text based on semantic relationships between questions and stored content.

Nevertheless, semantic retrieval is not universally superior in every situation. Exact identifiers, formal names, numerical values, and specialised terms may benefit from lexical

matching or structured filtering. A future version could combine semantic retrieval with keyword-based methods to improve coverage across different query types.

A direct performance comparison would require the same evaluation questions, equivalent knowledge sources, and clearly defined retrieval metrics for both approaches.

7.2.3 General Purpose Large Language Models

General-purpose LLMs can generate coherent answers to many natural-language questions. Their flexibility makes them useful for conversational interfaces, but their responses may include information that is outdated, incomplete, or unsupported by a particular application's approved knowledge.

The RAG architecture used by SmartGrama addresses this concern by providing retrieved information as context during generation. This creates a mechanism for connecting the answer to domain-specific material rather than relying only on the model's general training.

However, RAG does not eliminate hallucinations or guarantee that every answer is supported by evidence. Its effectiveness depends on retrieval quality, the completeness of the knowledge base, and the model's use of the supplied context. The implemented component should therefore be understood as a grounded-response approach rather than a guarantee of factual correctness.

7.2.4 Research on Retrieval Augmented Generation and Sentence Embeddings

Lewis et al. [3] introduced a widely cited RAG approach that combines a generative model with retrieved documents. Their work provides a conceptual foundation for systems that use external information during response generation.

Reimers and Gurevych [6] introduced Sentence-BERT, demonstrating the value of sentence embeddings for semantic similarity and information retrieval tasks. Their work is relevant to the use of sentence representations for matching user questions with knowledge-base content.

Research on multilingual sentence embeddings, including Reimers and Gurevych [7], is relevant to the design of language-independent or multilingual semantic-processing workflows. Work on Sinhala language models, including Ranasinghe et al. [2], also highlights the importance of language-specific evaluation when building systems for Sinhala.

These studies provide a theoretical and technical basis for the SmartGrama design choices. They do not, by themselves, demonstrate that the particular SmartGrama implementation achieves the same performance as the systems evaluated in those publications. Such a claim would require comparable datasets, evaluation methods, and experimental conditions.

7.2.5 Summary of the Comparison

Table 24 Qualitative Comparison of Conversational and Information Retrieval Approaches

Approach	Main Strength	Main Limitation	Relevance to SmartGrama
Rule-based chatbot	Predictable responses for predefined scenarios	Limited flexibility for varied questions	Useful for fixed workflows but less flexible for open-ended questions
Keyword-based retrieval	Direct matching of important terms	May miss semantically related wording	Could complement semantic retrieval
General-purpose LLM	Flexible natural-language generation	May generate unsupported information	Provides conversational response generation but requires grounding
Semantic retrieval	Matches content using vector representations	Depends on embeddings, indexed content, and retrieval settings	Supports finding relevant domain information
RAG-based assistant	Combines retrieval with contextual response generation	Depends on both retrieval and generation quality	Aligns with the component's intended design
SmartGrama NLP component	Integrates English–Sinhala processing, semantic retrieval, and generation for welfare and micro-loan information	Requires further quantitative and user-centred validation	Provides a domain-focused conversational interface within SmartGrama

The comparison is qualitative rather than the outcome of a controlled benchmark. The table should therefore be interpreted as a comparison of architectural characteristics, not as evidence that the SmartGrama component has achieved higher measured accuracy than competing systems.

7.3 Research Contribution

The research contribution of this work lies in applying and integrating established NLP and information-retrieval techniques within a multilingual welfare and micro-loan information-access context.

First, the component demonstrates a domain-focused application of RAG. Instead of treating the assistant as a general-purpose conversational tool, the design uses a knowledge base containing information relevant to the SmartGrama application. This supports a more focused approach to answering questions about welfare services and micro-loan information.

Second, the component combines semantic embeddings and vector retrieval with LLM-based generation. This integration creates a pipeline in which information is retrieved before the response is generated. It provides a basis for studying how retrieval relevance, context sufficiency, and answer grounding affect the quality of domain-specific responses.

Third, the work addresses English and Sinhala interaction within a single conversational component. This contributes an applied example of multilingual processing for an information access task in which terminology and factual consistency are important.

Fourth, the component includes an evaluation dataset of 217 question records. The dataset structure allows generated answers to be examined against expected answers and retrieved context, while response-time information supports the investigation of practical performance. This creates a foundation for more systematic evaluation and error analysis.

These contributions should be understood as an applied research and integration contribution. The work does not claim to have invented RAG, Sentence Transformers, FAISS, translation technology, or LLMs. Its contribution is the implementation and examination of these technologies within the defined SmartGrama domain.

7.4 Product Contribution

The product contribution concerns the functionality provided by the NLP component as part of the wider SmartGrama system.

The component is intended to provide a conversational interface through which users can submit questions about welfare and micro loan information in English or Sinhala. It connects the user interface to a backend processing pipeline that combines semantic retrieval, language processing, and response generation.

Its main product contributions are as follows:

1. **Natural language access:** Users can ask questions conversationally instead of navigating only through predefined menus or searching for exact keywords.
2. **Domain focused information retrieval:** The retrieval process searches the configured knowledge base for information relevant to the submitted question.
3. **Context assisted response generation:** Retrieved information is supplied to the LLM to guide the creation of the response.
4. **Multilingual interaction:** The component is designed to support English and Sinhala processing, helping accommodate users with different language preferences.
5. **Integration potential:** The component provides an NLP service that can be connected to the wider SmartGrama frontend and backend architecture.
6. **Evaluation support:** The structured question records allow developers to inspect generated answers, expected answers, retrieved context, and response times.

The product should be regarded as an informational assistant. It does not independently verify a user's identity, make official eligibility determinations, approve loans, or replace decisions made by authorised welfare agencies or financial institutions. Its usefulness depends on the quality, currency, and coverage of the information available to it.

7.5 Limitations and Threats to Validity

Although the component provides a technical foundation for multilingual welfare and micro-loan information access, several limitations must be considered when interpreting the findings.

7.5.1 Evaluation Dataset Limitations

The evaluation dataset contains 217 question records. While this supports structured technical inspection, the number of records alone does not establish that the dataset represents the full range of questions that real users may ask.

The dataset may not cover all variations in spelling, grammar, sentence structure, domain terminology, mixed-language usage, or unusual user requests. The available evidence also does not establish that the dataset was sampled randomly from a representative population of SmartGrama users.

Accordingly, conclusions drawn from the dataset should be limited to the questions and conditions actually evaluated. A larger and more diverse dataset would be required to strengthen claims about generalisation to real-world use.

7.5.2 Knowledge Base Coverage and Currency

The quality of a RAG based assistant depends heavily on the knowledge base. Missing documents, outdated procedures, inconsistent terminology, and incomplete descriptions can affect both retrieval and response generation.

Welfare requirements and micro-loan procedures may change over time. If the knowledge base is not updated, the assistant may provide information that no longer reflects current requirements. The use of retrieval cannot compensate for information that is absent or incorrect in the underlying source material.

A practical deployment should therefore establish a documented process for reviewing, updating, and approving domain information. Important information should be traceable to an appropriate source, and obsolete content should be removed or replaced.

7.5.3 Retrieval and Generation Errors

The component contains multiple processing stages, and an error at one stage can affect the final response. An embedding model may not represent a particular query effectively, the retrieval mechanism may return incomplete context, or the language model may misinterpret the retrieved material.

The documented error patterns involving unsupported URLs or procedures, incorrect terminology, and unsupported additional claims illustrate the need for explicit response-quality checks.

These limitations mean that the generated answer should not be treated as correct merely because it is fluent or because relevant context was retrieved. A more comprehensive evaluation should measure retrieval relevance, answer correctness, answer completeness, and factual support separately.

7.5.4 Translation and Sinhala Language Limitations

Translation can help connect English and Sinhala questions to the knowledge base, but the translation process may alter the meaning of domain-specific expressions. Terms relating to eligibility, financial obligations, documentation, and official procedures may not have simple one-to-one translations.

Sinhala input may also contain spelling variation, informal expressions, or mixed Sinhala-English wording. The available evidence does not establish that all these cases are handled reliably.

Future evaluation should include questions written naturally by Sinhala-speaking users and should involve competent bilingual reviewers. This would help identify whether translation or terminology affects the accuracy and clarity of the responses.

7.5.5 Lack of a Controlled Comparative Experiment

The current discussion compares the component conceptually with rule-based chatbots, keyword-based retrieval, general purpose LLMs, and established RAG approaches. The available evidence does not establish a controlled experiment in which these alternatives were tested on the same dataset under equivalent conditions.

Therefore, the work cannot claim a measured performance advantage over those alternatives. Future experiments should use a common test set, consistent evaluation criteria, and documented configurations to determine the relative strengths and weaknesses of each approach.

7.5.6 Limited User Centred Validation

Technical question and answer testing can reveal system errors, but it cannot fully establish usability, user trust, accessibility, or practical value. The available evidence does not establish that a statistically designed user study or representative stakeholder evaluation was conducted.

Users may interpret answers differently depending on their language proficiency, familiarity with digital systems, and understanding of welfare or financial terminology. A future user-centred study should examine whether participants can understand the answers, identify uncertainty, and recognise when they need to contact an official service provider.

7.5.7 Response Time and Deployment Constraints

The response time of the component depends on the development environment, model configuration, and workload. Results obtained in a local environment may not predict performance when several users access the system simultaneously.

A production deployment would require further assessment of concurrency, resource consumption, error recovery, service availability, security, and maintainability. Performance measurements should be collected under clearly documented conditions before making claims about production suitability.

7.5.8 Threats to Construct and External Validity

Construct validity concerns whether the selected evaluation measures genuinely represent the intended qualities of the assistant. For example, response time measures speed but does not measure factual correctness. Textual similarity between a generated answer and an expected answer may also fail to recognise a correct answer expressed differently.

External validity concerns whether the findings apply beyond the evaluated questions and environment. Since the documented dataset and local prototype do not establish broad real-world representativeness, the findings should not automatically be generalised to all English- or Sinhala-speaking users, all welfare programmes, or all micro-loan providers.

These limitations do not remove the value of the implementation. Rather, they define the boundaries of the conclusions that can responsibly be drawn and identify where further research is needed.

Table 25 Summary of Limitations and Recommended Responses

Limitation	Potential Effect	Recommended Response
Limited dataset representativeness	Some real-world questions may not be covered	Expand the dataset using diverse, reviewed questions
Incomplete or outdated knowledge	Incorrect or missing domain information	Establish regular source review and update procedures
Retrieval errors	Relevant information may not be provided to generation	Measure retrieval relevance and tune retrieval settings
Unsupported generated claims	Users may receive misleading details	Evaluate factual support and introduce verified safeguards
Translation and terminology issues	English and Sinhala answers may differ in meaning	Use bilingual review and language-specific testing
No controlled baseline comparison	Relative performance cannot be established	Conduct controlled experiments with common test data
Limited user-centred evidence	Usability and trust remain uncertain	Conduct structured testing with representative users
Local deployment conditions	Production performance may differ	Perform load, reliability, and security testing

7.6 Implications

The discussion has several implications for the design, evaluation, and future use of the SmartGrama NLP component.

7.6.1 Implications for System Design

The component demonstrates the importance of treating conversational AI as a pipeline rather than as a single language-model operation. Query processing, retrieval, context preparation, response generation, and output handling all contribute to the final result.

This architecture supports more systematic debugging because failures can be investigated at different stages. For example, an incorrect answer may result from missing knowledge, poor retrieval, translation problems, or unsupported generation. Recording these stages during evaluation can help developers identify the most appropriate corrective action.

7.6.2 Implications for Evaluation

The evaluation approach should measure multiple dimensions of quality. Correctness, completeness, factual support, retrieval relevance, language quality, and response time represent different properties and should not be reduced to a single unqualified score.

The existing 217-record dataset provides a starting point for this work. Calculating actual results from the dataset, documenting the evaluation procedure, and reporting the findings separately for English and Sinhala would strengthen the empirical contribution.

Where human reviewers are involved, their assessment criteria should be documented. Where automated metrics are used, their suitability and limitations should be explained. This would make the results easier to interpret and reproduce.

7.6.3 Implications for Responsible Use

Because the component provides information related to welfare services and micro-loans, reliability and transparency are particularly important. Users may rely on the assistant when trying to understand requirements or procedures. Unsupported statements can therefore create confusion even when they are not intended to replace official advice.

The assistant should clearly communicate the informational nature of its responses. It should avoid presenting unverified eligibility conclusions, financial commitments, official procedures, or website addresses as established facts. Where the available knowledge is insufficient, the preferred behaviour is to communicate the limitation and direct users towards an appropriate verified source, where one is available.

These safeguards should be implemented and tested rather than assumed to exist merely because the system uses RAG.

7.6.4 Implications for Future Development

The component can be improved through a number of practical directions. These include expanding and reviewing the knowledge base, improving retrieval for domain-specific terminology, evaluating alternative embedding models, and testing approaches that combine semantic and keyword-based retrieval.

Multilingual improvements should prioritise the preservation of meaning across English and Sinhala. Domain terminology should be reviewed consistently, and bilingual evaluation should identify cases in which translated questions or answers differ in important details.

The response-generation process could also be strengthened through evidence checks, clearer handling of insufficient context, and systematic testing for unsupported claims. Finally, deployment testing should examine resource consumption, concurrent access, response time, and operational reliability.

These improvements should be prioritised according to measured failure patterns rather than assumed benefits. Evaluation evidence can therefore serve not only as a final assessment but also as an ongoing mechanism for improving the component.

8: COMMERCIALISATION AND SUSTAINABILITY

8.1 Value Proposition and Target Users

The SmartGrama Natural Language Processing (NLP) component is designed to improve access to welfare and micro-loan information through a conversational assistant that supports English and Sinhala. Its potential value lies in enabling users to ask questions in natural language and receive responses generated using information retrieved from a domain-specific knowledge base.

Traditional information-access methods may require users to navigate multiple web pages, search through documents, contact service providers, or understand formal descriptions of application procedures. A conversational assistant offers an alternative interaction method by allowing users to express their information needs as questions.

The NLP component combines semantic retrieval, language-processing functionality, and Large Language Model (LLM) based response generation through a Retrieval Augmented Generation (RAG) architecture. These technologies provide a technical foundation for a system that can retrieve potentially relevant welfare and micro-loan information and present it in a conversational format.

The component's commercialisation potential should be considered in relation to its current development stage. The existing implementation provides a foundation for further development, but commercial deployment would require additional validation, verified information sources, operational support, and appropriate arrangements with organisations responsible for the information being provided.

8.1.1 Value Proposition

The main value proposition of the SmartGrama NLP component is to make welfare and micro-loan information easier to access through multilingual, natural language interaction.

The component may offer the following benefits:

1. **Convenient information access:** Users can submit questions conversationally instead of relying exclusively on manual document searches or predefined navigation options.
2. **English and Sinhala interaction:** Supporting both languages may help users who prefer to communicate in either English or Sinhala.
3. **Domain-focused responses:** The use of a dedicated knowledge base provides a mechanism for connecting generated answers to relevant welfare and micro-loan information.
4. **Reduced information search effort:** Users may be able to locate relevant information through a single interface rather than searching across multiple sources.
5. **Potential support for service providers:** Organisations may use a suitably validated version of the assistant to support routine information enquiries, subject to their operational requirements.
6. **Extensibility:** The component's modular architecture provides a basis for future improvements to retrieval, language processing, response generation, and integration with other SmartGrama services.

These benefits represent the intended value of the component. Their actual impact on user effort, service-provider workload, or information accessibility would need to be demonstrated through further evaluation.

8.1.2 Target Users

The potential target users can be grouped into direct users and organisations that may benefit from providing conversational information access.

Table 26 Potential Target Users and Their Expected Benefits

Target Group	Information Needs	Potential Benefit
Welfare information seekers	Information about welfare services, requirements, and application procedures	Easier access to relevant information
Micro-loan information seekers	General information about micro-loan processes, requirements, and repayment-related matters	More convenient access to financial-service information
English-speaking users	Information requested in English	Natural-language interaction in English
Sinhala-speaking users	Information requested in Sinhala	Improved convenience for users who prefer Sinhala
Community service organisations	Repeated questions about services and requirements	Potential support for routine information enquiries

Target Group	Information Needs	Potential Benefit
Welfare and financial-service organisations	A possible additional digital information channel	Potential improvement to information delivery, subject to approval and validation
SmartGrama administrators	Management and improvement of the NLP component	A foundation for maintaining and extending the assistant

The table identifies potential users rather than confirmed customers or partners. The needs of each group should be validated through stakeholder interviews, user testing, or a structured requirements study before commercial decisions are made.

8.2 Market and Competitive Position

The commercialisation potential of the SmartGrama NLP component depends on whether the proposed functionality addresses a meaningful information-access need and whether it can be delivered reliably within an appropriate operational environment.

The component operates within the broader area of conversational artificial intelligence, information retrieval, and multilingual digital services. Its distinguishing design characteristic is the application of these technologies to welfare and micro-loan information in English and Sinhala.

This section presents a qualitative market position assessment. No formal market-size study, competitor pricing analysis, or customer-demand survey has been established by the implementation evidence currently available. Therefore, the discussion identifies potential positioning rather than claiming a verified market opportunity.

8.2.1 Potential Market Need

Welfare and micro-loan information can involve eligibility requirements, application procedures, supporting documents, service conditions, and other domain-specific details. Users may find it difficult to identify the most relevant information when it is distributed across different documents or presented using formal terminology.

A multilingual conversational assistant may help address this problem by allowing users to ask questions in familiar language and receive a response based on retrieved information.

However, the existence and scale of this need should be validated rather than assumed. Relevant evidence could be collected through interviews with potential users, consultations with welfare and financial-service organisations, and observation of existing information-access processes.

8.2.2 Comparison with Alternative Solutions

Potential alternatives include static information websites, frequently asked questions (FAQ) pages, rule-based chatbots, manual support channels, and general-purpose AI assistants.

Table 27 Qualitative Competitive Positioning

Alternative	Strength	Limitation or Trade-Off	Potential Position of SmartGrama NLP
Static websites and documents	Can present detailed, authoritative information	Users must locate and interpret relevant content	Provides a conversational route to relevant information
FAQ pages	Efficient for common and predictable questions	May not cover questions phrased differently from the listed questions	Uses natural-language questions and semantic retrieval
Rule-based chatbots	Predictable responses for defined workflows	Can require extensive rules to support varied questions	Offers more flexible question handling through retrieval and generation
Manual support channels	Can provide context-sensitive assistance	May require staff time and operating resources	Could support routine enquiries while preserving escalation to human support
General-purpose AI assistants	Flexible conversational responses	May not be grounded in the organisation's approved information	Uses a domain-specific knowledge base to guide responses
SmartGrama NLP component	Designed for English–Sinhala welfare and micro-loan information access	Requires further quality, usability, and deployment validation	Offers a domain-focused multilingual information-access approach

This comparison describes architectural and functional differences rather than measured competitive superiority. The assistant may complement existing websites and human support services rather than replace them.

8.2.3 Differentiation Strategy

The component's proposed differentiation is based on four characteristics.

First, it is designed for a specific information domain rather than unrestricted conversation. This allows the knowledge base and evaluation process to focus on welfare and micro-loan enquiries.

Second, it supports English and Sinhala processing, subject to language-specific quality validation. This provides a basis for addressing users with different language preferences.

Third, it uses a RAG architecture to connect answer generation with retrieved domain information. This creates an opportunity to improve the relevance and traceability of responses compared with generation that does not use a domain-specific retrieval stage.

Fourth, it is developed as a component of the wider SmartGrama system. This provides a potential route for integration with other application functions, subject to interface compatibility and integration testing.

These characteristics form a proposed differentiation strategy. Their practical importance should be tested through user feedback and comparisons with the information-access methods currently used by target organisations.

8.2.4 Market Validation Requirements

Before the component is commercialised, the following questions should be investigated:

- Which groups experience the greatest difficulty accessing welfare and micro-loan information?
- Which types of questions occur most frequently?
- Do users prefer a conversational interface to existing websites, FAQs, or manual assistance?
- Is English–Sinhala support sufficient for the intended user population?
- Which organisations would be willing to provide, review, and maintain the knowledge base?
- What accuracy, response time, security, and availability requirements would an organisation impose?
- Would an organisation fund development, hosting, maintenance, or integration?

Answers to these questions would help determine whether the component should be developed as an institutional service, a community information tool, a licensed software product, or a non commercial digital service.

8.3 Business or Sustainability Model

The sustainability of the SmartGrama NLP component depends on maintaining the software, keeping the knowledge base accurate, supporting users, and ensuring that the technical infrastructure remains operational.

A sustainable model must consider both financial and non-financial requirements. Even if the software is initially developed as an academic prototype, continued operation may involve costs for computing resources, hosting, technical maintenance, information review, and quality assurance.

No confirmed commercial arrangement, revenue stream, or funding agreement is established by the available implementation evidence. The models below should therefore be treated as possible future options rather than existing business activities.

8.3.1 Potential Sustainability Models

Table 28 Potential Business and Sustainability Models

Model	Description	Potential Advantages	Main Considerations
Institutional licensing	An organisation pays to use and maintain a customised deployment	Provides a possible source of recurring income	Requires customer demand, support arrangements, and licensing terms

Model	Description	Potential Advantages	Main Considerations
Software-as-a-Service (SaaS)	The assistant is offered through a hosted service	Centralised maintenance and potential access across multiple organisations	Requires secure hosting, operational monitoring, and cost management
Institutional partnership	An organisation supports development and operation for its users	May align the assistant with verified institutional information	Requires formal responsibilities and an agreed governance model
Grant-funded or development-funded service	Development and maintenance are supported by grants or institutional funding	Can support public-benefit use without charging end users	Funding may be time-limited and requires ongoing financial planning
Open-source or community-supported model	Selected code or resources are made available for collaborative development	Can encourage transparency and external contributions	Requires governance, maintenance ownership, and licence review
Internal operational tool	The assistant is maintained within an organisation's existing digital services	May support routine information enquiries	Benefits and savings need to be measured before investment decisions

The most suitable model cannot be selected solely from the technical architecture. It should depend on validated demand, operational costs, institutional requirements, ownership arrangements, and the intended public-benefit objectives.

8.3.2 Recommended Initial Sustainability Approach

For the current development stage, a phased institutional or partnership-oriented approach is a reasonable option to investigate. The first priority should be to validate the assistant with a limited set of approved information sources and a clearly defined group of users.

A possible sequence is:

1. **Prototype validation:** Verify that the assistant answers representative questions appropriately and identify recurring retrieval, generation, and language errors.
2. **Stakeholder consultation:** Discuss the proposed functionality with relevant welfare or financial service organisations and identify requirements for approved information, privacy, and operational responsibility.
3. **Limited pilot:** If an appropriate partner and verified content are available, conduct a controlled pilot with a defined scope and documented evaluation criteria.
4. **Operational assessment:** Estimate hosting, model inference, maintenance, information review, and support costs based on actual usage.
5. **Sustainability decision:** Select an appropriate funding or service model using evidence from the pilot, stakeholder feedback, and the cost assessment.

This sequence reduces the risk of committing to a commercial model before the component's practical value and operational requirements have been established.

8.3.3 Cost Considerations

The main cost categories likely to affect future operation are summarised in Table 8.`14.

Table 29 Potential Operational Costs

Cost Category	Description	Cost Control Consideration
Hosting and computing	Resources required to run the backend and language model	Measure resource use under realistic workloads
Model inference	Computing resources consumed when generating responses	Evaluate model size, response length, and deployment options
Knowledge-base maintenance	Reviewing, correcting, and updating domain information	Establish a documented update and approval process
Software maintenance	Fixing defects and updating dependencies	Maintain version control and a scheduled maintenance process
Testing and quality assurance	Testing accuracy, multilingual behaviour, and reliability	Prioritise high-risk and frequently asked questions
Security and privacy	Protecting user interactions and system access	Apply appropriate data minimisation, access control, and retention policies
User support	Handling reports, corrections, and operational questions	Define responsibilities and escalation procedures
Deployment and integration	Connecting the component to organisational systems	Estimate effort from actual integration requirements

Actual cost estimates should be based on the chosen deployment model, infrastructure, usage levels, and maintenance responsibilities. No financial projections should be presented as established results without supporting cost and demand data.

8.3.4 Non-Financial Sustainability

Financial sustainability alone is insufficient for a public-information assistant. The component also requires:

- **Information sustainability:** Content must remain accurate, relevant, and appropriately approved.
- **Technical sustainability:** Software dependencies, model configurations, and deployment procedures must remain maintainable.
- **Operational sustainability:** Responsibilities for incident handling, user feedback, and updates must be clearly assigned.
- **Social sustainability:** The assistant should remain understandable and accessible to the intended users.
- **Trust sustainability:** Users should be informed about the assistant's limitations and the distinction between informational guidance and official decisions.

These requirements are essential to maintaining the component's usefulness over time.

8.4 Scalability, Intellectual Property and Compliance

8.4.1 Technical Scalability

The component uses a modular architecture involving a frontend, backend processing, semantic retrieval, a knowledge base, language-processing functionality, and LLM-based response generation. This separation provides a foundation for future extension.

Potential scalability improvements include increasing knowledge-base coverage, improving retrieval settings, selecting suitable embedding models, and adapting the inference infrastructure to expected demand.

However, modularity does not automatically guarantee production scalability. A system that operates correctly for one user in a local development environment may require significant changes to handle concurrent requests, high availability, or large amounts of domain information.

Before deployment at scale, the component should be assessed for:

- Concurrent request handling.
- Average and upper percentile response time.
- CPU, memory, and other resource requirements.
- Retrieval performance as the knowledge base grows.
- Recovery from backend, model, or network failures.
- Monitoring, logging, and error reporting.
- Secure access to administrative functions.
- Consistent performance for English and Sinhala queries.

Scalability decisions should be based on measured demand and documented technical requirements.

8.4.2 Knowledge Base Scalability

The knowledge base may need to grow as additional welfare programmes, application procedures, and micro-loan information sources are introduced.

Increasing the amount of stored information creates a need for content governance. Documents should be reviewed for relevance, accuracy, duplication, and currency before being incorporated into the system. Information that becomes obsolete should be updated or removed.

A future knowledge management process should record the source of each document, its review date, its approval status, and the person or organisation responsible for maintaining it. This would support traceability and make it easier to investigate incorrect responses.

The retrieval configuration should also be re-evaluated as the knowledge base expands. Additional content may introduce overlapping passages or information that is relevant to a different programme. Testing should establish whether the retrieval process continues to return suitable evidence for representative questions.

8.4.3 Intellectual Property Considerations

The commercialisation of the NLP component requires clarity regarding intellectual property ownership and permitted use.

The implementation combines software code, open-source libraries, language models, embeddings, and domain information. These elements may be governed by different licence terms and usage conditions.

Before commercial deployment, the project team should review the applicable terms for all significant dependencies and model resources. The review should cover software redistribution, commercial use, attribution, model-specific restrictions, and any conditions attached to third-party data.

The ownership of original project code and documentation should also be established according to the applicable institutional and project arrangements. The right to use welfare or financial-service documents should not be assumed merely because those documents are publicly accessible.

Table 30 Intellectual Property and Licensing Considerations

Component	Consideration	Recommended Action
Original source code	Ownership and permission to distribute or commercialise	Confirm ownership and applicable project agreements
Third-party libraries	Licence conditions and attribution	Maintain a dependency and licence inventory
Embedding model	Model licence and permitted usage	Verify the exact model and its terms
LLM and inference software	Model and software usage conditions	Review the selected model and deployment terms
Knowledge-base documents	Copyright, reuse permission, and update rights	Use approved sources and record permissions
Project documentation	Ownership and permitted distribution	Confirm the applicable institutional requirements

This table identifies areas requiring review. It does not constitute a legal determination that every dependency or source has already been approved for commercial use.

8.4.4 Privacy, Security and Responsible Use

Although the component is designed to provide information, users may submit personal details in their questions. Such information could include contact details, application-related information, or other sensitive content.

The deployment design should therefore minimise unnecessary collection and retention of personal information. Where conversation logs are needed for testing or improvement, their purpose, access permissions, retention period, and handling procedures should be defined.

Security measures should include appropriate backend access controls, protection of configuration secrets, dependency maintenance, and validation of user inputs. If external services are used for translation, model inference, or hosting, the information transmitted to those services should be reviewed before deployment.

The assistant should also communicate its role clearly. It should not represent itself as an official government or financial institution unless that status has been formally established. It should not present generated information as a confirmed eligibility decision or loan approval.

Any production deployment should undergo a compliance review appropriate to its intended use, data flows, organisational setting, and applicable legal obligations. The available project evidence does not establish that a formal legal or security audit has already been completed.

8.4.5 Operational Governance

Long-term operation requires clear responsibility for approving information, maintaining the software, reviewing user feedback, and responding to identified errors.

For example, a future institutional deployment could assign separate responsibilities for technical maintenance and domain content approval. Technical maintainers would manage the software and retrieval pipeline, while authorised subject-matter representatives would review changes to official welfare or financial service information.

This separation would reduce the risk of technical changes inadvertently altering approved domain content. It would also provide a clearer process for correcting inaccurate or outdated responses.

Before a pilot begins, the operating organisation should define how users can report problems, how high-risk errors are escalated, and how corrections are incorporated into the knowledge base and evaluation dataset.

8.5 Contribution to the Sustainable Development Goals

The SmartGrama NLP component has potential relevance to selected United Nations Sustainable Development Goals (SDGs) because it is designed to improve access to welfare and micro-loan information. This relationship should be described as a potential contribution rather than a demonstrated social impact.

8.5.1 SDG 1: No Poverty

SDG 1 focuses on ending poverty in all its forms. Access to reliable information about welfare services may help individuals understand available support and the requirements associated with relevant programmes.

The SmartGrama NLP component may contribute indirectly by providing a more convenient way to search for welfare information. However, providing information does not itself establish that users receive benefits, experience reduced poverty, or achieve improved financial security.

A stronger assessment would examine whether users can identify relevant programmes more effectively and whether the assistant helps reduce barriers to accessing accurate information.

8.5.2 SDG 8: Decent Work and Economic Growth

SDG 8 concerns inclusive economic growth, productive employment, and decent work. Access to accurate information about micro-loan procedures may help users understand the information required when exploring financial services.

The assistant may support this objective indirectly by making relevant information easier to discover. It does not provide credit, guarantee access to finance, determine creditworthiness, or establish that a loan is appropriate for a particular user.

Any claimed contribution should therefore focus on information accessibility rather than suggesting that the system directly produces employment or economic growth outcomes.

8.5.3 SDG 9: Industry, Innovation and Infrastructure

SDG 9 includes the development of resilient infrastructure, inclusive and sustainable industrialisation, and innovation. The SmartGrama NLP component applies established NLP and information-retrieval technologies to a domain-specific digital service.

Its potential relevance to SDG 9 lies in demonstrating how conversational AI and retrieval-based architectures can be integrated into an information-access application. Further development, testing, and deployment could provide an example of applied digital innovation.

The implementation should not be interpreted as evidence of broad infrastructure impact without further deployment and outcome data.

8.5.4 SDG 10: Reduced Inequalities

SDG 10 addresses inequality within and among countries. Supporting English and Sinhala interaction may reduce some language related barriers to information access for the intended user groups.

However, language support alone does not guarantee equitable access. Digital literacy, device availability, connectivity, disability accessibility, language quality, and the accuracy of available information can also affect whether users benefit from the service.

Future evaluation should examine whether the assistant is usable by people with different language preferences and levels of digital experience.

8.5.5 SDG Alignment Summary

Table 31 Potential Alignment of the SmartGrama NLP Component with the SDGs

Sustainable Development Goal	Potential Relationship	Evidence Needed to Assess Impact
SDG 1: No Poverty	Easier access to welfare-service information	User studies on information discovery and access to relevant services
SDG 8: Decent Work and Economic Growth	Easier access to micro-loan information	Evaluation of information comprehension and usefulness
SDG 9: Industry, Innovation and Infrastructure	Applied use of conversational AI and semantic retrieval	Deployment evidence, reliability measures, and documented technical outcomes
SDG 10: Reduced Inequalities	English–Sinhala information access	Language-specific quality results and accessibility testing

The proposed SDG alignment is based on the component's intended purpose. Demonstrating measurable social impact would require evidence beyond technical implementation and question-level testing.

9: CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

This research focused on the design, implementation, and evaluation of a Natural Language Processing (NLP) component for the SmartGrama application. The component was developed to improve access to welfare and micro loan information through a multilingual conversational assistant supporting English and Sinhala.

Accessing welfare and micro-loan information can require users to search through documents, interpret formal descriptions, and identify the requirements relevant to their questions. The proposed component addresses this information-access problem by providing a conversational interface through which users can submit natural-language questions and receive generated responses informed by a domain-specific knowledge base.

The implemented architecture combines a frontend interface, a Flask backend, Sentence Transformers for generating text embeddings, FAISS for similarity-based retrieval, translation functionality provided through `deep-translator`, and Ollama for local Large Language Model (LLM) inference. These technologies form a Retrieval-Augmented Generation (RAG) workflow in which relevant information is retrieved and supplied as context to guide response generation.

The use of semantic retrieval provides a mechanism for identifying information based on the meaning of a question rather than relying exclusively on exact keyword matching. The generation stage then uses the retrieved context to produce a conversational response. This approach is suitable for the intended domain because welfare and micro-loan questions may involve specific procedures, eligibility requirements, supporting documents, and repayment-related information.

The component was also examined using a structured evaluation dataset containing 217 question records. The available records include questions, expected answers, generated answers, retrieved context, language information, categories, and response times. This dataset provides a foundation for investigating answer quality, retrieval behaviour, multilingual processing, and response time performance.

The evaluation material also identifies important limitations, including unsupported URLs or procedures, incorrect terminology, and additional claims that are not supported by retrieved evidence. These issues demonstrate that the use of RAG does not automatically guarantee correct or complete answers. The reliability of the component depends on the quality of the knowledge base, the relevance of retrieved information, the behaviour of the language model, and the effectiveness of the evaluation and error-handling processes.

The findings should therefore be interpreted within the scope of the implemented prototype and the available technical evidence. The documented dataset does not, by itself, establish a particular accuracy percentage, a statistically significant improvement over alternative systems, or a measurable increase in user satisfaction. Further quantitative analysis and user-centred evaluation are necessary before making broader performance claims.

Overall, the research provides a technical foundation for a domain focused English Sinhala conversational assistant for welfare and micro loan information. Its main contribution is the application and integration of established NLP, semantic retrieval, translation, and language-

generation technologies within the SmartGrama system. Further development should focus on improving factual grounding, multilingual quality, knowledge-base maintenance, evaluation coverage, and operational reliability.

9.2 Achievement of Research Objectives

The research objectives established in Chapter 1 provide the basis for assessing the work completed in the individual NLP component. The objectives are considered in relation to the documented implementation and evaluation evidence rather than unsupported performance assumptions.

9.2.1 Objective 1: Investigate the Challenges of Accessing Welfare and Micro-Loan Information

The first objective was to investigate the information access challenges relevant to the SmartGrama domain and identify how NLP techniques could support users seeking welfare and micro-loan information.

The research addressed this objective by examining the need for natural-language interaction, semantic information retrieval, multilingual processing, and context-informed response generation. These requirements informed the design of the component and the selection of the RAG architecture.

The research also recognised that domain specific information must be accurate, understandable, and sufficiently complete to be useful. This established the need to assess not only whether a response was generated, but also whether it addressed the question using relevant and supported information.

9.2.2 Objective 2: Design a RAG Based NLP Architecture

The second objective was to design an NLP architecture that connects semantic retrieval with LLM-based response generation.

This objective was addressed through the design of a processing pipeline involving user-query handling, embedding generation, similarity-based retrieval, context preparation, and response generation. Sentence Transformers and FAISS provide the technical basis for semantic retrieval, while Ollama supports local LLM inference.

The architecture was designed to connect the conversational interface with the domain knowledge base and backend processing services. It provides a structured approach to retrieving relevant information before generating an answer.

The design establishes a technical mechanism for context-assisted generation, although further evaluation is required to determine how consistently the retrieved evidence is reflected in the final responses.

9.2.3 Objective 3: Implement English and Sinhala Processing

The third objective was to implement multilingual functionality to support English and Sinhala interaction.

The component incorporates translation functionality through `deep-translator` as part of its language-processing workflow. This provides a basis for processing queries and generating responses for users who prefer either language.

The implementation represents progress towards the multilingual objective. However, successful integration of translation functionality should not be interpreted as proof of equivalent answer quality in both languages. Further testing with representative English and Sinhala questions, including bilingual review of technical terminology, is required to establish language-specific performance.

9.2.4 Objective 4: Integrate the Component with the SmartGrama Application

The fourth objective was to integrate the NLP functionality within the SmartGrama application architecture.

The component uses a frontend–backend arrangement in which the user interface submits questions for processing and receives generated responses. The backend connects the relevant language processing, retrieval, and generation functions.

The documented development evidence includes local backend activation and embedding-model loading. These observations support the technical feasibility of the local processing workflow. However, the extent of production readiness must be assessed separately through deployment, reliability, security, and integration testing.

9.2.5 Objective 5: Evaluate the Component

The fifth objective was to evaluate the implemented NLP component using structured question-and-answer testing.

A dataset containing 217 question records was used to examine questions, expected answers, generated answers, retrieved context, language categories, and response times. This provides a structured basis for identifying response quality problems and investigating the behaviour of the retrieval and generation stages.

The available evidence identifies failure patterns involving unsupported procedural details, URLs, terminology, and claims that are not supported by retrieved information. These findings provide useful directions for improving the system.

However, the available material does not establish final aggregate accuracy, precision, recall, language specific performance percentages, or statistically validated comparisons with alternative systems. Completing the objective quantitatively requires calculating suitable metrics from the evaluation records and documenting the assessment procedure.

Table 32 Summary of Research Objectives and Evidence of Achievement

Research Objective	Evidence of Work Completed	Remaining Requirement
Investigate welfare and micro-loan information-access challenges	Problem analysis and domain-specific requirements	Further stakeholder validation
Design a RAG-based NLP architecture	Architecture connecting retrieval, context preparation, and generation	Further testing of retrieval and grounding quality
Implement English and Sinhala processing	Translation functionality and multilingual processing workflow	Language-specific evaluation and bilingual review
Integrate the NLP component	Frontend–backend workflow and documented local execution evidence	Comprehensive integration and deployment testing
Evaluate the NLP component	Structured dataset containing 217 question records	Calculation and reporting of quantitative metrics and broader validation

The table summarises the evidence available for each objective. It should be updated if additional test reports, screenshots, measured results, or formal stakeholder-validation records become available before submission.

9.3 Individual Research and Product Contributions

The work contributes to SmartGrama through both a research contribution and a practical software component.

9.3.1 Individual Research Contribution

The research contribution is the investigation and application of a RAG based NLP approach for English Sinhala access to welfare and micro-loan information.

The component brings together established technologies to support a domain-specific conversational workflow. It provides a practical context for examining the relationship between semantic retrieval, retrieved context quality, translation, and LLM-based answer generation.

The structured evaluation dataset further provides a basis for investigating the quality of generated answers and identifying recurring error patterns. By retaining expected answers and retrieved context alongside generated responses, the evaluation process can support a more detailed investigation of system behaviour than simply checking whether a response was returned.

The research also identifies the limitations of the selected approach. Retrieval may fail to return sufficient information, translation may affect meaning, and the LLM may generate unsupported claims. These limitations highlight the importance of evaluating the separate stages of a conversational AI pipeline.

The research does not claim to introduce a new embedding algorithm, vector database, translation method, or language model. Its contribution is the application, integration, and evaluation of these techniques within the SmartGrama information access context.

9.3.2 Individual Product Contribution

The product contribution is the development of an NLP component intended to provide a conversational information-access facility within SmartGrama.

The main product elements include:

- A conversational interface for submitting natural-language questions.
- Backend processing for handling requests and coordinating NLP operations.
- Semantic embedding generation using Sentence Transformers.
- Similarity-based retrieval using FAISS.
- Translation functionality supporting the English Sinhala workflow.
- Context-assisted response generation through an Ollama based LLM.
- A structured evaluation dataset for investigating response quality and technical behaviour.

Together, these elements provide a foundation for further development of the SmartGrama conversational assistant.

The component remains informational in scope. It is not an official welfare eligibility assessment system, a loan approval mechanism, an identity-verification service, or a credit-risk assessment engine. Its purpose is to help users discover and understand relevant information, while official decisions remain the responsibility of the appropriate organisations.

9.3.3 Contribution to the Wider SmartGrama System

Within the wider SmartGrama application, the NLP component can act as the conversational information access layer. Its modular architecture provides a potential connection between the user interface, the knowledge base, and other application services.

The component may be extended in the future to support additional verified information sources or to improve the way retrieved information is presented. Such extensions would require clearly defined interfaces, appropriate data permissions, and integration testing.

The contribution is therefore both technical and functional: it provides a basis for adding multilingual natural-language interaction to the wider SmartGrama system.

9.4 Future Directions

Although the component establishes a foundation for multilingual RAG-based information access, further work is required to improve reliability, usability, and deployment readiness. The following directions are recommended according to their relevance to the identified limitations.

9.4.1 Improve Knowledge Base Quality and Maintenance

The first priority is to improve the quality, completeness, and currency of the knowledge base.

The assistant can only retrieve information that is available and appropriately represented in its knowledge sources. Missing or outdated documents may therefore result in incomplete answers even when the retrieval and generation components operate as intended.

Future development should introduce a structured knowledge-management process that records document sources, review dates, approval status, and update responsibilities. Information relating to eligibility, procedures, supporting documents, and repayment conditions should be reviewed by appropriate subject-matter representatives before being used to inform responses.

An update and re-indexing process should also be documented so that changes to approved content are reflected in the retrieval system.

9.4.2 Improve Retrieval Relevance

Further work should investigate whether the retrieval stage consistently identifies the most relevant information for different question types.

The evaluation dataset can be used to identify questions for which the retrieved context does not contain the information needed to produce a correct answer. These cases should be classified to determine whether the problem results from query formulation, embedding quality, knowledge base coverage, retrieval configuration, or other factors.

Potential improvements include testing alternative embedding models, adjusting retrieval parameters, introducing metadata filters, and combining semantic retrieval with keyword-based matching.

These changes should be evaluated against the same test questions before and after modification. This would make it possible to determine whether the changes improve retrieval quality rather than relying only on qualitative impressions.

9.4.3 Strengthen Factual Grounding and Response Handling

A further priority is to reduce unsupported claims in generated answers.

Future versions should investigate methods for checking whether substantive statements are supported by the retrieved context. The assistant should also be tested on questions for which the knowledge base contains insufficient information.

Where adequate evidence is unavailable, the desired behaviour is to acknowledge the limitation instead of producing a definitive but unsupported response. For important procedural details, official website addresses, and eligibility requirements, the system should rely on verified sources.

Potential safeguards should be treated as development tasks until their behaviour has been implemented and validated. Testing should specifically include unsupported URLs, incorrect procedures, conflicting information, and questions outside the intended domain.

9.4.4 Improve English Sinhala Processing

The multilingual workflow should be evaluated separately for English and Sinhala.

Future testing should include naturally written Sinhala questions, variations in spelling, informal expressions, mixed Sinhala-English wording, and specialised welfare or financial terminology. Bilingual reviewers should examine whether the generated answers preserve the intended meaning and communicate the same substantive information across both languages.

A terminology guide could be developed for recurring concepts such as eligibility requirements, supporting documents, application procedures, and repayment conditions. This would help maintain consistency across the knowledge base and generated responses.

Additional language models or multilingual embedding approaches could be investigated if measured results demonstrate that the current configuration is insufficient. Model selection should be based on evaluation evidence, licensing conditions, and deployment requirements.

9.4.5 Expand Quantitative Evaluation

The existing dataset of 217 question records provides a foundation for more detailed performance analysis.

Future evaluation should report clearly defined metrics for answer correctness, completeness, retrieval relevance, factual support, and response time. Where appropriate, results should be separated by language and question category.

A practical evaluation procedure could include:

1. Checking the completeness and validity of all evaluation records.
2. Establishing explicit criteria for judging generated answers against expected answers.
3. Reviewing retrieved context to determine whether it contains the required evidence.
4. Calculating appropriate metrics using the documented evaluation records.
5. Reporting results separately for English and Sinhala where sufficient data are available.
6. Investigating errors through qualitative analysis.
7. Repeating evaluation after significant system changes.

If a larger dataset is collected, it should include varied question wording and examples of questions that cannot be answered from the knowledge base. The dataset should also be reviewed for duplication and possible overlap between development and final testing records.

Any quantitative results should state the evaluation method, sample size, model configuration, and limitations. This would improve the transparency and reproducibility of the research.

9.4.6 Conduct User Centred Evaluation

Technical evaluation should be supplemented by a structured assessment involving representative users.

Participants could be asked to complete realistic information search tasks using the assistant and an appropriate alternative, such as an existing information page or FAQ. Evaluation could examine task completion, time required to locate information, answer comprehension, perceived usefulness, and user-reported problems.

Participants who prefer English and those who prefer Sinhala should both be represented. The study should also consider differences in digital literacy and familiarity with welfare or financial service terminology.

Any user study should follow the applicable institutional ethics and consent requirements. Its results would provide evidence about practical usability and perceived value that cannot be established solely through automated question-and-answer testing.

9.4.7 Improve Performance and Deployment Readiness

The local implementation provides a starting point for further deployment work, but additional testing is necessary before the component can be considered suitable for broad operational use.

Future work should measure response times under documented hardware and model configurations. It should also examine concurrent requests, resource consumption, service interruptions, and recovery from failures.

Other priorities include secure configuration management, access controls for administrative functions, appropriate logging and retention practices, dependency maintenance, and operational monitoring.

The deployment architecture should be selected according to the intended usage volume, available infrastructure, data-handling requirements, and financial constraints. Production suitability should be established through testing rather than inferred from successful local execution.

Table 33 Prioritised Future Development Directions

Priority	Future Direction	Main Purpose	Suggested Evidence of Progress
High	Knowledge-base review and update process	Improve information accuracy and currency	Approved source inventory and documented update records
High	Factual grounding and insufficient-context handling	Reduce unsupported responses	Evaluation of unsupported-claim and unanswerable-question cases
High	Quantitative analysis of the 217-record dataset	Establish evidence-based performance results	Reproducible metric calculations and error analysis

Priority	Future Direction	Main Purpose	Suggested Evidence of Progress
High	English–Sinhala quality evaluation	Identify language-specific issues	Separate evaluation results and bilingual review
Medium	Retrieval optimisation	Improve relevance and context sufficiency	Before-and-after retrieval evaluation
Medium	User-centred evaluation	Assess usability and perceived value	Structured user-study results
Medium	Deployment and load testing	Establish operational requirements	Documented response-time, resource, and reliability results
Medium	Privacy and security review	Reduce operational and data-handling risks	Documented review and tested safeguards
Future	Controlled expansion to additional domains or languages	Increase coverage	Approved content and evaluation results for each extension
Future	Continuous monitoring and regression testing	Support long-term quality improvement	Documented error-review and release-testing process

9.5 Final Conclusion

The SmartGrama NLP component addresses the problem of providing convenient access to welfare and micro-loan information through a multilingual conversational interface. By integrating Sentence Transformers, FAISS, translation functionality, a domain specific knowledge base, and Ollama based LLM inference, the component establishes a RAG-based processing workflow for English and Sinhala questions.

The work demonstrates the application of established NLP and information-retrieval technologies within a defined information access problem. The structured evaluation dataset containing 217 question records provides a basis for investigating the relationship between expected answers, retrieved context, generated responses, and response times. The documented failure patterns also identify areas where further development and validation are necessary.

The principal contribution is the implementation and examination of a domain-focused conversational NLP component within the SmartGrama system. Its intended value is to make information discovery more convenient while providing a mechanism for grounding generated responses in relevant domain information.

Nevertheless, the component should not be considered a replacement for official welfare agencies or financial-service providers. Further work is needed to establish quantitative performance, improve the reliability of English Sinhala responses, validate usability with representative users, maintain verified information sources, and assess production deployment requirements.

In conclusion, the SmartGrama NLP component provides a foundation for further development of multilingual, retrieval supported information services. Its long term success will depend on the combination of reliable technology, accurate and current knowledge, transparent limitations, systematic evaluation, and responsible operational management.

REFERENCES

[1] T. B. Brown et al., “Language Models are Few-Shot Learners,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.

[2] S. Dissanayake et al., “Enhancing Conversational AI Model Performance and Explainability for Sinhala-English Bilingual Speakers,” in *Proceedings of the International Conference on Advancements in Computing (ICAC)*, 2022, doi: 10.1109/ICAC57685.2022.10025153.

[3] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.

[4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2019, pp. 4171–4186.

[5] Y. Gao et al., “Retrieval-Augmented Generation for Large Language Models: A Survey,” arXiv preprint arXiv:2312.10997, 2023.

[6] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3982–3992.

[7] J. Johnson, M. Douze, and H. Jégou, “Billion-scale Similarity Search with GPUs,” arXiv preprint arXiv:1702.08734, 2017.

[8] N. Reimers and I. Gurevych, “Making Monolingual Sentence Embeddings Multilingual using Knowledge Distillation,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 4512–4525.

[9] D. Jurafsky and J. H. Martin, *Speech and Language Processing*, 3rd ed., online manuscript.

[10] S. Ranasinghe et al., “Sinhala Encoder-only Language Models and Evaluation,” in *Proceedings of the 2025 Annual Meeting of the Association for Computational Linguistics*, 2025, doi: 10.18653/v1/2025.acl-long.422.